Data Structures and Algorithms Lecture – 8 Sorting Methods - Part A

M.G.Noel A.S Fernando (PhD)

Professor

UCSC

Sorting Methods

- Arranging things into ascending or descending order is called sorting.
- Or A Sorting Algorithm is used to rearrange a given array or list elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of elements in the respective data structure.

  i.e. Sorting is the process of arranging items in order.

Some important factors that must be considered are:

- **Speed** : the simplest algorithms are $O(N^2)$ while more advanced ones are O(N log N). No algorithm can make less than O(N log N) comparisons between keys.

- **Storage** : algorithms that sort in place are the best, needing memory O(N). Those are using linked list representation need extra N words of memory for references and those that work on a copy of the file needed O(2N).

- **<u>Simplicity</u>** : the simpler algorithms are often easiest to implement and outer perform more sophisticated once for small problem.
- **<u>Stability :</u>** an algorithm is stable iff it preserve the relative order of records with equal keys. there are ways to convert unstable implementation into stable ones.

Examples

g e e k s f o r g e e k s =====> e e e e f g g k k o r s s

Input Output

Sorting methods are traditionally classified as:

- **Internal :** these take advantage of fast, random access, but only acceptable when the whole file fit in to the main memory.
- **External** : large file must be stored using secondary storage devices. The rate of data transfer between primary and secondary memory is usually bottleneck.

Sorting Techniques

There are three major sorting techniques. Such as

- Sorting by selection
- Sorting by insertion
- Sorting by exchange

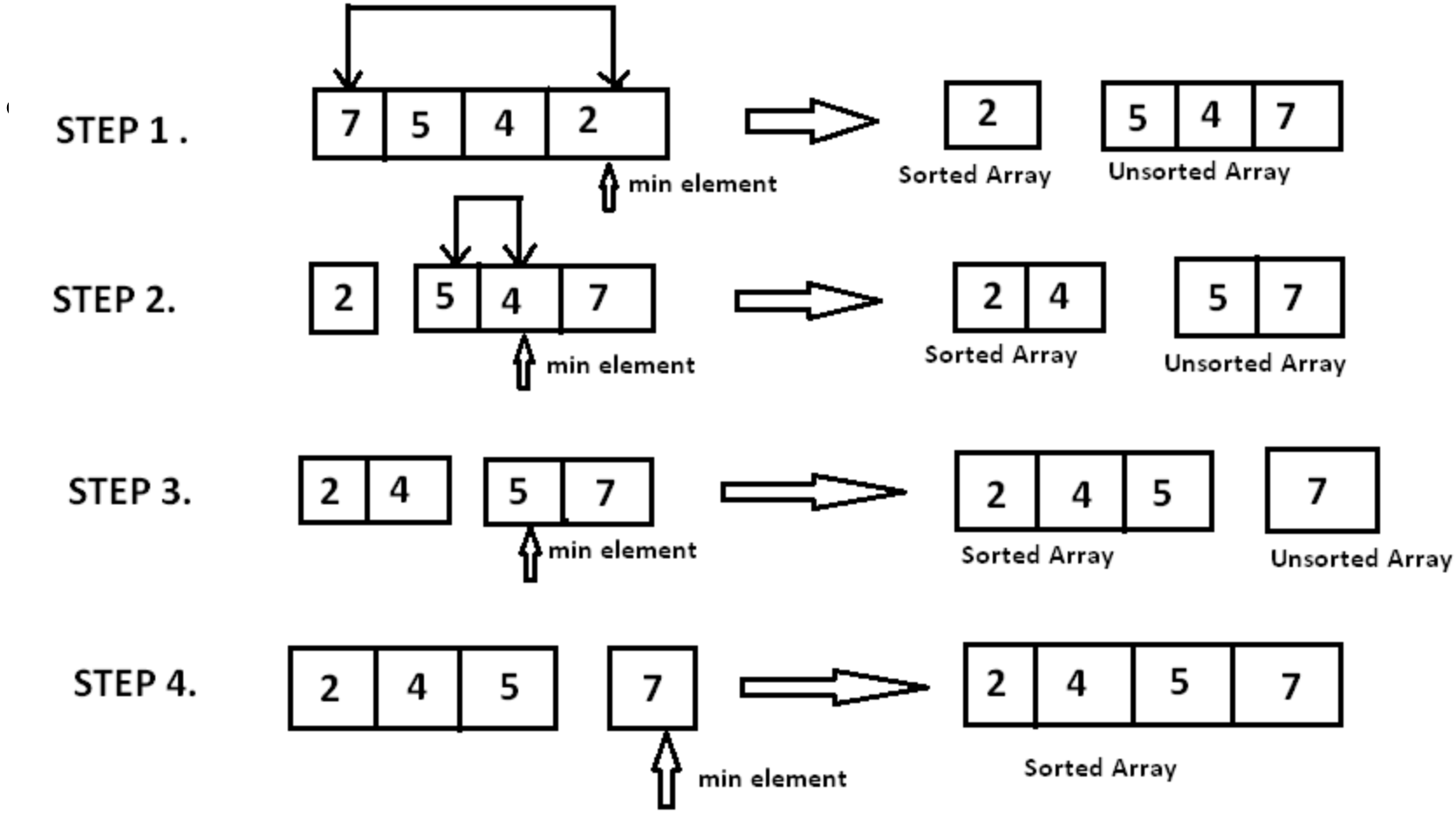
Sorting by Selection

- **E.g. Selection Sort**
- In terms of an array A, the selection sort finds the smallest element in the array and exchanges it with A[0].
- Then, ignoring A[0], the sort finds the next smallest and swaps it with A[1] and so on.

Demo

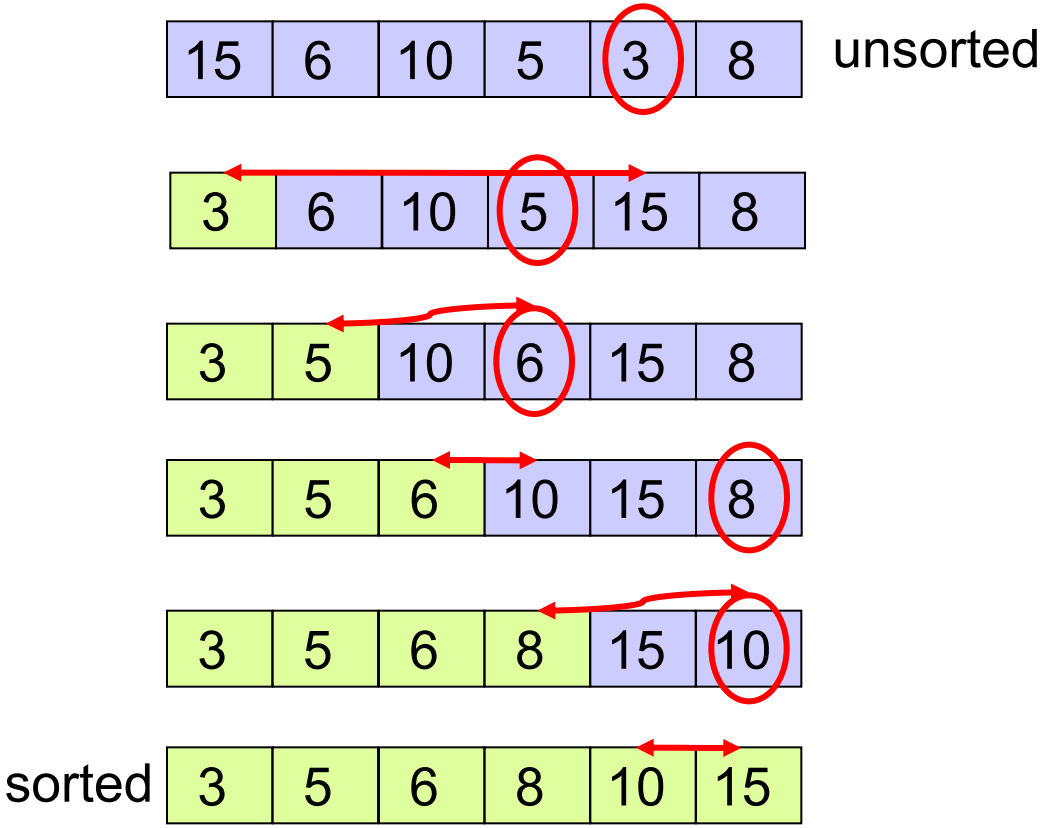
- https://www.youtube.com/watch?v=LriMvv9qDrk

Selection Sort



Selection Sort

- An example of selection sort



Iterative Selection Sort

- Iterative selection sort algorithm

```
//Sort the first n elements of an array
Input: array A, int n
Output:
selectionSort1(A, n){
        for(index = 0; index<n-1; index++){
            indexOfNextSmallest = the index of the smallest value
             among A[index], A[index+1], …A[n-1]
             interchange the value of A[index] and A[indexOfNextSmallest]
        }
}
```

Selection Sort – Pseudocode

Input: An array $A[1..n]$ of n elements.
Output: $A[1..n]$ sorted in descending order

```
for i ← 1 to n - 1
  min ← i
  for j ← i + 1 to n   {Find the i th smallest element.}
      if A[j] < A[min] then
          min ← j
  end for
  if min ≠ i then interchange A[i] and A[min]
end for
```

Selection sort code

```
function selectionSort(array){
  for(var i = 0; i < array.length; i++){
    var min = i;
    for(var j = i + 1; j < array.length; j++){
      if(array[j] < array[min]){
        min = j;
      }
    }
    if(i !== min){
      var temp = array[i];
      array[i] = array[min];
      array[min] = temp;
    }
  }
  return array
}

selectionSort([34,22,10,19,17])
```

Time Complexities of Selection Sort

- Worst Case Time Complexity is: $O(N^2)$
- Average Case Time Complexity is: $O(N^2)$
- Best Case Time Complexity is: $O(N^2)$

Recursive Selection Sort

Recursive selection sort algorithm

```
//Sort the array elements A[first] through A[last] recursively
Input: array A, int first, int last
Output:
selectionSort2(A, first, last){
    if(first<last){
        indexOfNextSmallest = the index of the smallest value
among
                A[first], A[first+1], …A[last]
        interchange the value of A[first] and
A[indexOfNextSmallest]
        selectionSort2(A, first+1, last);
    }
}
```

Running Time Analysis

- Selection sort is $O(n^2)$ regardless of the initial order of the elements in an array.
- Ex : discuss Best, average and worst case time complexities.

Sorting by insertion

- Insertion sort Algorithm
- An insertion sort algorithm starts by considering the first two element of the data array.
- Which is data[0] and data[1]. If they are out of order, then shift data[0] by one position and data[1] can insert into empty position.
- Then consider the third element ,(i.e. data[2]). If this element is smaller than data[1] and data[0] shift data[0] and data[1] by one position and insert data[2] into that location and so-on

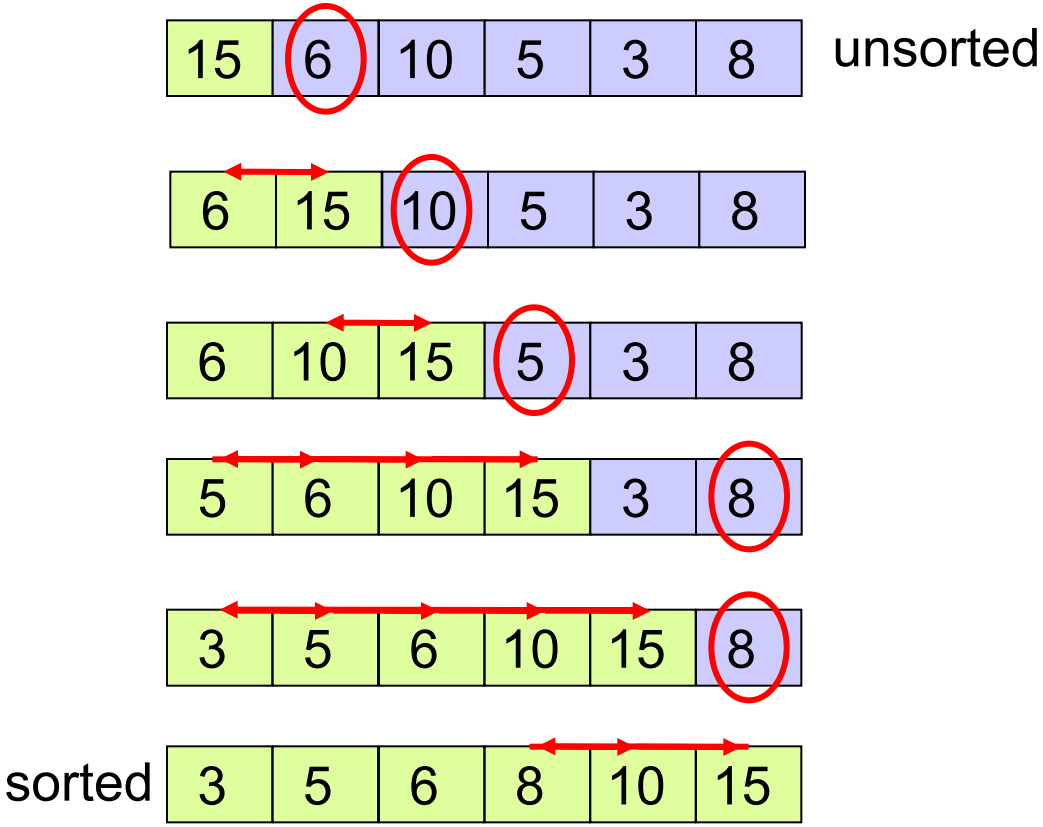
Insertion Sort

- An insertion sort of an array partitions the array into two parts.
  - One part is sorted and initially contains just the first element in the array.
  - The second part contains the remaining elements.

  The sort inserts one by one the elements in the unsorted part of the array into their proper location within the sorted part of the array.

Insertion Sort

- An example of selection sort



Pseudocode algorithm -Insertion Sort

```
Step 1 – If it is the first element, it is already sorted. return 1;
Step 2 – Pick next element
Step 3 – Compare with all elements in the sorted sub-list
Step 4 – Shift all the elements in the sorted sub-list that is greater than
         value to be sorted
Step 5 – Insert the value
Step 6 – Repeat until list is sorted
```

Pseudo code algorithm for insertion sort

```
Insertionsort (data[]){
for (i=0;i<data.length-1;i++)
temp=data[i];
Move all elements data[j]  grater than temp
   by one position;
Place temp in its proper position;
```

Each element data[I] is inserted into its proper position j such that $0 \leq j \leq i$

Code for the Insertion Sort

```c
#include <math.h>
#include <stdio.h>

// Insertion Sort Function
void insertionSort(int array[], int n)
{
    int i, element, j;
    for (i = 1; i < n; i++) { element = array[i]; j = i - 1; while (j >= 0 &&
            array[j + 1] = array[j];
            j = j - 1;
        }
        array[j + 1] = element;
    }
}

// Function to print the elements of an array
void printArray(int array[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", array[i]);
    printf("n");
}
```

Insertion Sort

- Running time analysis
  - Insertion sort is at best O(n) and at worst $O(n^2)$.
  - The closer an array is to sorted order, the less work an insertion sort does

Sorting by Exchange

- Eg : Bubble sort
- One of the simplest sorting is algorithm is known as bubble sort. The algorithm as follows.
- Beginning at the last element in the list (or start from the first element from the list)
- Compare each element with the previous element in the list. If an element is less than its predecessor, swap these two element.
- To completely sort the list, you need to perform this process n-1 times on a list of length n

- Eg :
- (a)Run through the bubble sort algorithm by hand on the list :44,55,12,42,94,18,06,67
- (b) Write a C program to implementing the bubble sort algorithm on an array.
- The basic idea underlying the bubble sort is to pass through the file sequentially several times.
- Each pass consists of comparing each element in the file with its predecessor [x[j] and x[j-1] and interchanging the two elements if they are not proper order.

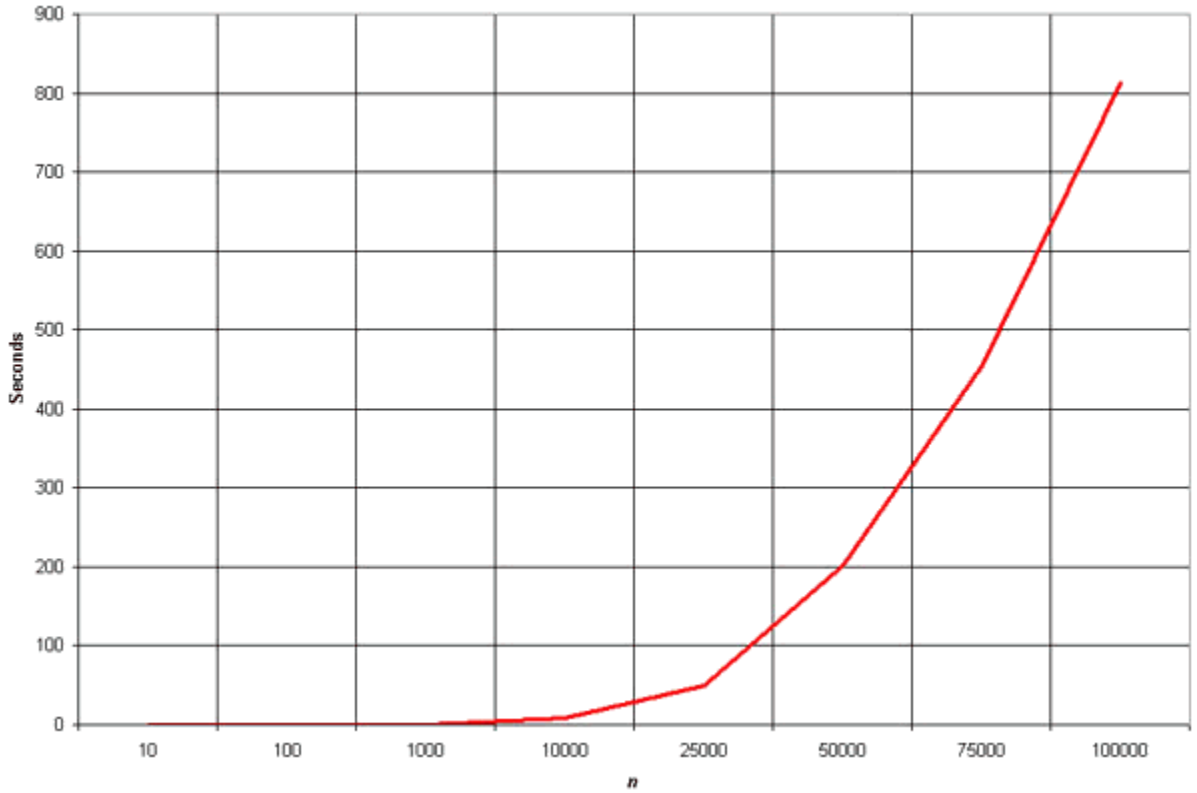
- The following comparisons are made on the first pass
- 44,55,12,42,94,18,06,67
- Data[7] with Data[6] (67,06) No interchange
- Data[6] with Data[5] (06,18) interchange
- Data[5] with Data[4] (06,94) interchange
- Data[4] with Data[3] (06,42) interchange
- Data[3] with Data[2] (06,12 interchange
- Data[2] with Data[1] (06,55) interchange
- Data[1] with Data[0] (06,44) interchange
- after the first pass, the smallest element is in its proper positions.
- **06,44,55,12,42,94,18,67**

- Original -> 44,55,12,42,94,18,06,67
- Pass 1 -> **<u>06</u>**,44,55,12,42,94,18,67
- Pass 2 -> **<u>06,12</u>**,44,55,18,42,94,67
- Pass 3 -> **<u>06,12,18</u>**,44,55,42,67,94
- Pass 4 -> **<u>06,12,18,42</u>**,44,55,67,94
- Pass 5 -> **<u>06,12,18,42,44</u>**,55,67,94
- Pass 6 -> **<u>06,12,18,42,44,55</u>**,67,94
- Pass 7 -> **<u>06,12,18,42,44,55,67</u>**,94
- Sorted file->**<u>06,12,18,42,44,55,67</u>,<u>94</u>**

Pseudo Code Algorithm

- **SEQUENTIAL BUBBLESORT (A)**
- for $i \leftarrow 1$ to length do
  for $j \leftarrow$ length downto i +1 do
  If $A[j] < A[j\text{-}1]$ then
  Exchange $A[j] \leftrightarrow A[j\text{-}1]$
- Here the number of comparison made
  $1 + 2 + 3 + \ldots + (n - 1) = n(n - 1)/2$
  $= O(n^2)$

Running time -Bubble Sorting Algorithm



Pseudo code Algorithm

- bubblesort(data[])
- for (i=0,i<data.length-1;i++)
- for (j=data.length-1;j>i;- - j)
- swap elements in positions j and j-1 if they out of order;

Time complexity of bubble sort

- The number of comparison is same in each case (best, average and worst case) and equals the total number of iterations of the inner for loops:

$$\sum_{i=0}^{n-2}(n-1-i) = n(n-1)/2 = O(n^2)$$

Lecture – 8
Advanced Sorting Methods –Part B

M.G.Noel A.S Fernando (PhD)

Professor

Dept. of Information Systems Engineering, UCSC

Sorting Algorithms

- We learned basic algorithms, such as sorting by exchange (e.g., Bubble Sort), sorting by insertion (e.g., Insertion Sort), and sorting by selection (e.g., Selection Sort).
- In advanced sorting algorithms, we study two main categories of sorting techniques.
- **Comparison-Based Sorting Algorithms** (e.g., Merge Sort, Shell Sort, Quick Sort, Heap Sort).
- **Non-Comparison-Based Sorting Algorithms** (e.g., Counting Sort, Radix Sort, Bucket Sort).

Merge Sort

- **Type:** Comparison-based, Divide-and-Conquer algorithm
- Procedure :Merge Sort is a **divide-and-conquer sorting algorithm**. It works by repeatedly splitting the list into smaller parts, sorting those parts, and then merging them back together in sorted order.

Method 1- Steps of Merge Sort:

Step 1: Divide

- Split the unsorted list into two halves (left and right).
- Keep dividing until each sublist has only one element (a single element is already sorted).

Step 2: Conquer (Sort recursively):

- Recursively apply merge sort to each half.

Step 3: Merge:

- Combine (merge) the two sorted halves into a single sorted list.
- This is done by repeatedly comparing the smallest elements from each half and appending the smaller one.

Algorithm:

mergeSort(arr[], l, r)

If l < r

1. Find the middle point to divide the array into two halves: middle m = (l+r)/2

2. Call mergeSort for first half: Call mergeSort(arr, l, m)

3. Call mergeSort for second half: Call mergeSort(arr, m+1, r)

4. Merge the two halves sorted in step 2 and 3: Call merge(arr, l, m, r)

Queston 1

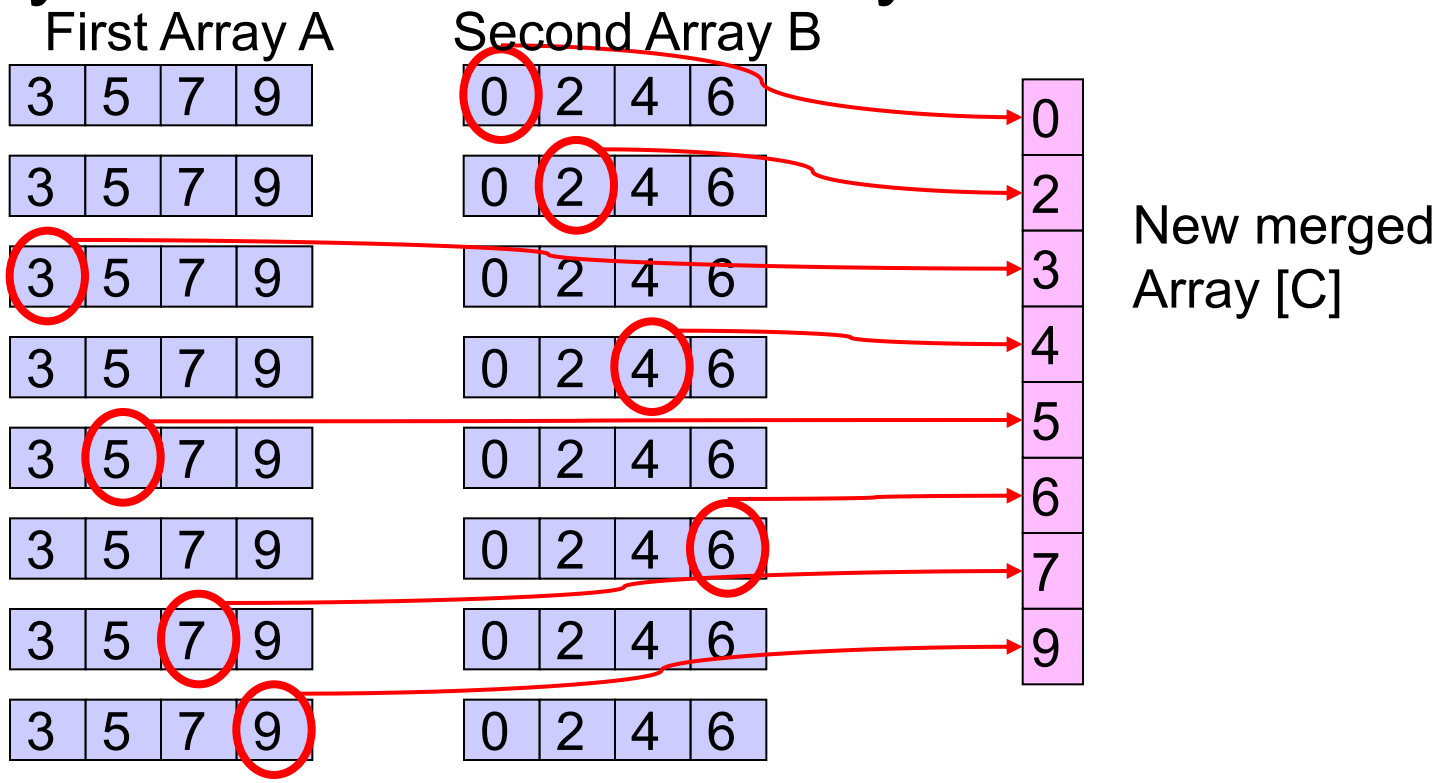
- Consider the following data set:
- 38,27,43,3,9,82,10
- Using a suitable diagram, explain how to sort the above dataset using the **Merge Sort algorithm**.
- Write C code to implement the Merge Sort algorithm

Method 2-Merge Sort

Merge sort Algorithm

- **<u>Merging</u>**
- Merging is the process of combining two or more sorted file into a third sorted file.
- The basic merge algorithm takes two input arrays A and B, output array C, and three counters Actr,Bctr,Cctr which is initially set the beginning of their respective arrays.
- The smaller of A[Actr],B[Bctr] is copy to the next entry in C, and the appropriated counters are advanced. When either input array is exhausted, the rest of the other array is copy to C.

Merge sort

- Merge function — Merging two sorted arrays into one sorted array.
- 

Merge Sort (Cont..)

- If the array A contains 3,5,7,9 and Array B contain 0,2,4,6, then the algorithm proceed as follows :
- First comparison is done between 0 and 3, then 0 is added to C ,then 2 and 3 , then 2 is added etc.

Merge sort - pseudo code Algorithm

```
/* An and Bn are the maximum number of elements of array A and B.*/
Actr:=0;
Bctr:=0;
Cctr:=0;
While (Actr<=A_n-1) and (Bctr<= B_n-1)
{
If A[Actr] < B[Bctr] {
C[Cctr]=A[Actr]
Actr++;
}
else
```

Merge sort - pseudo code Algorithm(cont..)

```
C[Cctr]=B[Bctr]
Bctr++;
}
Cctr++;
}
/* copy the remaining elements
While Actr < A_n-1
/* Copy remaining elements from B */
{
C[Cctr]=A[Actr];
Actr++;
Cctr++;
}
```

Merge sort - pseudo code Algorithm(cont..)

```
While Bctr<= B_n-1
/* copy remaining elements from B */
{
C[Cctr]= B[Bctr];
Bctr++;
Cctr++;
}
}
```

Straight Merge Sort

- Divide the file into n sub files of size 1 and merge adjacent pairs of files.
- Then we have approximately n/2 files of size 2.
- Repeat this process until there is only one file remaining of size n.

Example

25,57,48,37,12,92,86,33

Original file is divided into n sub-files.

[25],[57],[48],[37],[12],[92],[86],[33]

Pass 1

[25,57],[37,48],[12,92],[33,86]

Pass 2

[25,37,48,57],[12, 33, 86 92]

Pass 3

[12,25,33,37,48,57,86,92]

Entire file is sorted.

Shell Sort

- Shell Sort is an **in-place comparison-based sorting algorithm**.
- It is basically an improvement of **Insertion Sort** that allows the exchange of far-apart elements to reduce the total number of shifts.
- In Insertion Sort, elements are moved only one position at a time → slow for large, unsorted arrays.
- Shell Sort improves this by comparing and swapping elements that are far apart.
- This method sorts separate sub-files of the original file. These sub-files contained K element of the original file.
- The value of k is called increment or gap .

Shell Sort (Cont..)

- For example,
- Original file : x[1], x[2], x[3], x[4],....., x[15]
- If k=5 then
- Sub file 1 : x[1],x[6],x[11].....
- Sub file 2 : x[2],x[7],x[12].....
- Sub file 3 : x[3],x[8],x[13].....
- Sub file 4 : x[4],x[9],x[14].....
- Sub file 5 : x[5],x[10],x[15].....
- There are five sub-files, each containing one fifth elements of the original file.

Shell Sort (Cont..)

- In general I^{th} element of the Jth sub file is x[(i-1)*k+j)]
- For example, if the original file is 25,57,48,37,12,92,86,33 and the sequence of increments (4,2,1) are chosen.
- For the first iteration (Increment = 4)
- We can divide the original file into four sub files.

Shell Sort (Cont..)

- 25,57,48,37,12,92,86,33
- 1st sub file (x[1],x[5]) [25,12]
- 2nd sub file (x[2],x[6]) [57,92]
- 3rd sub file (x[3],x[7]) [48,86]
- 4th sub file (x[4],x[8]) [37,33]

- 12,57,48,33,25,92,86,37

Second iteration (increment=2)

- 12,57,48,33,25,92,86,37
- First sub file :(x[1],x[3],x[5],x[7])
- (12, 48,25,86)
- Second sub file ::(x[2],x[4],x[6],x[8])
- (57, 33,92,37)
- [12,33,25,37,48,57,86,92)

Second iteration (increment=1)

- [12,33,25,37,48,57,86,92)
- Increment =1
- [12,25,33,37,48,57,86,92]
- Sorted file.

Pseudocode Algorithm (Shell Sort with increments/ Gaps 4,2,1)

```
Algorithm ShellSort(A, n)
Input: Array A of length n
Output: Sorted array A
gap_sequence = [4, 2, 1]
for each gap in gap_sequence do
   for i = gap to n-1 do
      temp = A[i]
      j = i
      while j >= gap and A[j-gap] > temp do
         A[j] = A[j-gap]
         j = j - gap
      end while
      A[j] = temp
   end for
end for
```

Lecture – 8
Advanced Sorting Methods –Part C

M.G.Noel A.S Fernando (PhD)

Professor

Dept. of Information Systems Engineering, UCSC

Quick Sort Algorithm

- Quick sort is best on the method of sorting by exchange.
- This is an improved version of bubble sort and is a lot faster, on average than the bubble sort.
- This algorithm was invented in 1960 and it is still the best sorting algorithm known for sequential computers.

Quick sort is :

- Easy to implement
- Consumes few recourses than any other sorting methods in many situations.

It works by

- partitioning the file into two sub-files
- Sorting these two sub files independently in recursive way.

Quicksort

- The quick sort algorithm also uses the divide and conquer strategy.
- The quick sort partitions the sequence by dividing it into two segments based on a selected pivot key.
- In addition, the quick sort can be implemented to work with virtual subsequences without the need for temporary storage.

Quicksort

- **Quicksort** is a Divide and Conquer algorithm.
- It picks an element as a pivot and partitions the given array around the picked pivot.
- There are many different versions of quickSort that pick pivot in different ways.
  - pick the first element as a pivot.
  - pick the last element as a pivot
  - Pick a random element as a pivot.
  - Pick median as the pivot.

Quicksort

- The quick sort is a simple recursive algorithm that can be used to sort keys stored in either an array or list.
- Given the sequence, it performs the following steps:

Quicksort

- The first key is selected as the pivot, p. The pivot value is used to partition the list into two segments or subsequences, L and U, such that L contains all keys less than the p and U contains all keys greater than or equal to p.
- The algorithm is then applied recursively to both L and U. The recursion continues until the base case is reached, which occurs when the sequence contains fewer than two keys.
- The two segments and the pivot value are merged to produce a sorted sequence.
- This is accomplished by copying the keys from segment L back into the original sequence, followed by the pivot value and then the keys from segment U.
- After this step, the pivot key will end up in its proper position within the sorted sequence.

Quicksort

Quicksort divides an array/list into two pieces. These pieces are not necessarily halves of the array. It chooses one element in the array — called the pivot (p) and rearranges the array/list elements so that

- Elements in positions before the pivot are less than the pivot
- Elements in positions after the pivot are larger than or equal to the pivot

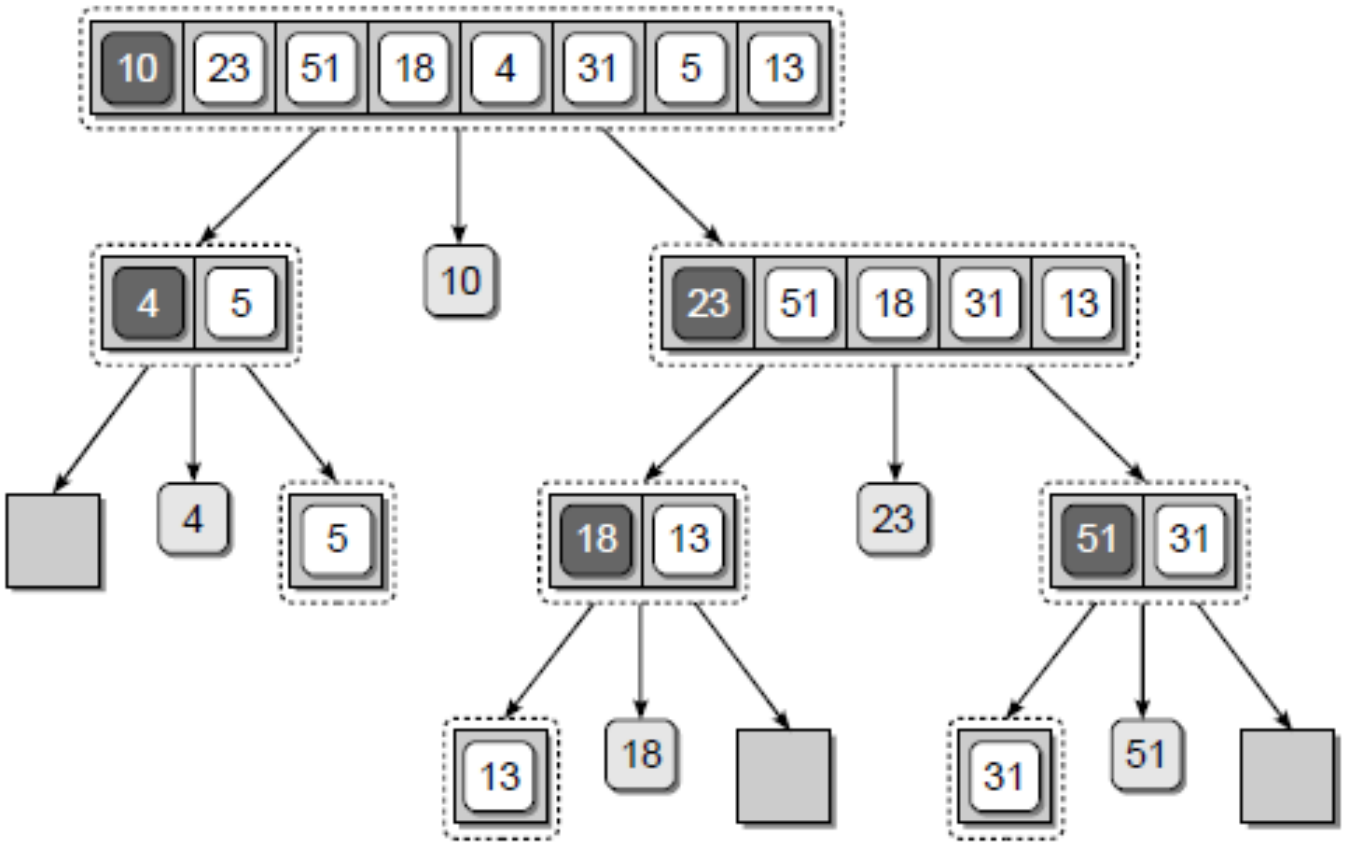
The arrangement is called a partition of the array/list.

Partition

- Let x be an array/list and n be the number of elements in the array to be sorted.
- Choose a element **a** (pivot) from a specific positions within the array (e.g **a** can be chosen as the first element so that a=x[0])
- Suppose that the element of x are partitioned so that **a** is placed into position J and the following conditions are hold:

1. Each of the elements in positions 1 through j-1 is less than to **a**
2. each of the elements in positions j+1 to n is grater than of equal to **a**

Example



Question 2

a)Sort the following dataset using the **Quick Sort algorithm**:
25, 57, 48, 37, 12, 92, 86, 33

- **Note:** Use the **first element** as the pivot at all stages.

b) write pseudocode algorithm for the quick sort

Partition - Algorithm

- Let a=x[lb] be the element whose final position is sought.
- Two references **up** and **down** are initialized to the upper and lower bounds of the sub-array respectively.
- At any point during the execution, each element in a position above **up** is grater than **a.**

Partition Algorithm

- Two references **up** and **down** are moved towards each other in the following fashion.
- **Algorithm**
- **Step 1**: Repeatedly increase the pointer **down** by one position until **x[down] >a**
- **Step 2**: Repeatedly decrease the pointer **up** by one position until x[up] =< a
- **Step 3**: if up > down then interchange **x[down] with x[up]**
- The process is repeated until the condition in step 3 fails (ie up =< down) at that point **x[up]** is interchanged with **x[lb]** (which equal to a), whose final position was sought and **j** is set to **up**.

Question 3

- Apply the above portion algorithm portion the following data set:
- 25 57 48 37 12 92 86 33

Lecture – 8
Advanced Sorting Methods –Part D

M.G.Noel A.S Fernando (PhD)

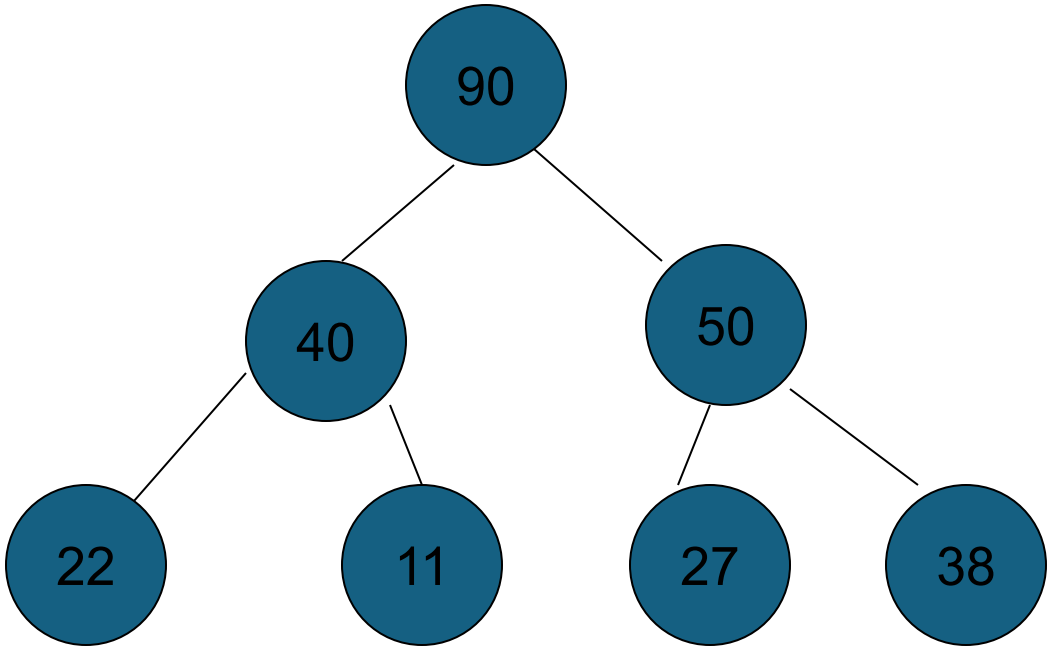
Professor

Dept. of Information Systems Engineering, UCSC

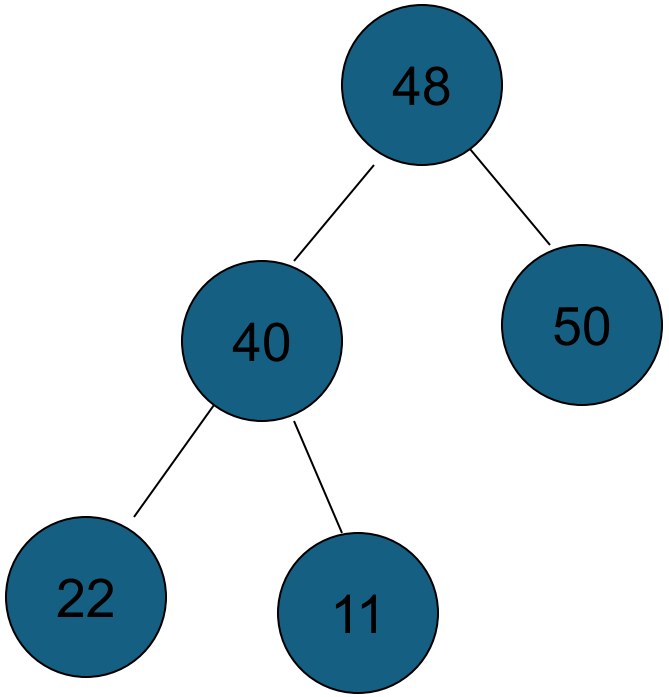
Heaps, Heap sorts and Priority Queues

- Heap
- A heap of size n is a B-tree of n nodes such that the values of each node is >= the value of its children. (if any)

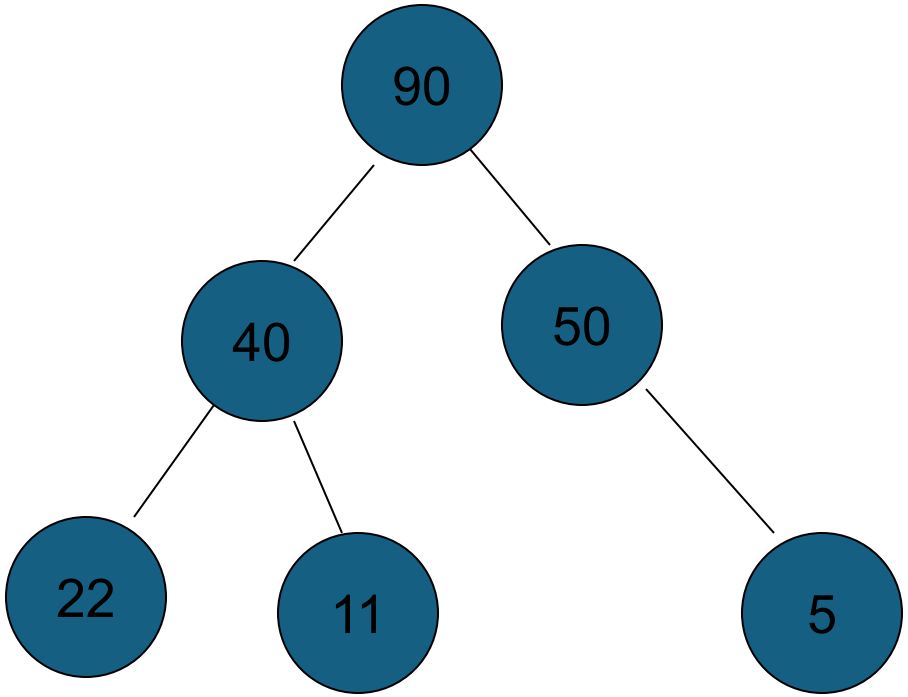
Examples



A Heap



Not a heap [ie 50 <48]



Not a Heap [Gaps]

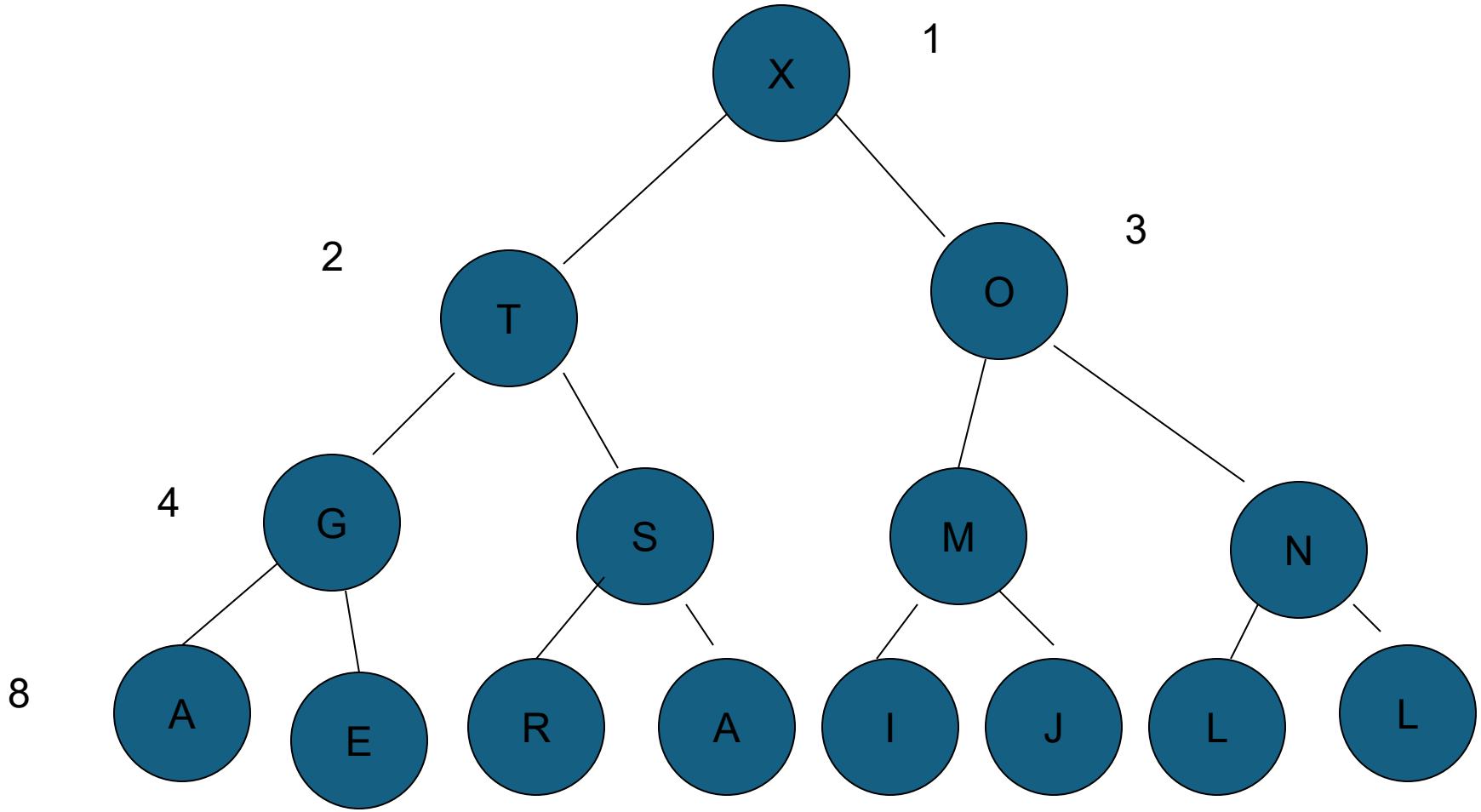
Properties

- A heap, however, is a special kind of binary tree that leaves no gaps in an array implementation.
- The definition of the heap data type is as follows.

A heap is a binary tree satisfying the following restrictions.

- All leaves are on adjacent levels.
- All leaves on the lowest level occur at the left of the tree
- All levels above the lowest are completely filled.
- Both children of any node are again heaps.
- The values stored at any node is at least as large as the values in its two children.

Array representation of a heap

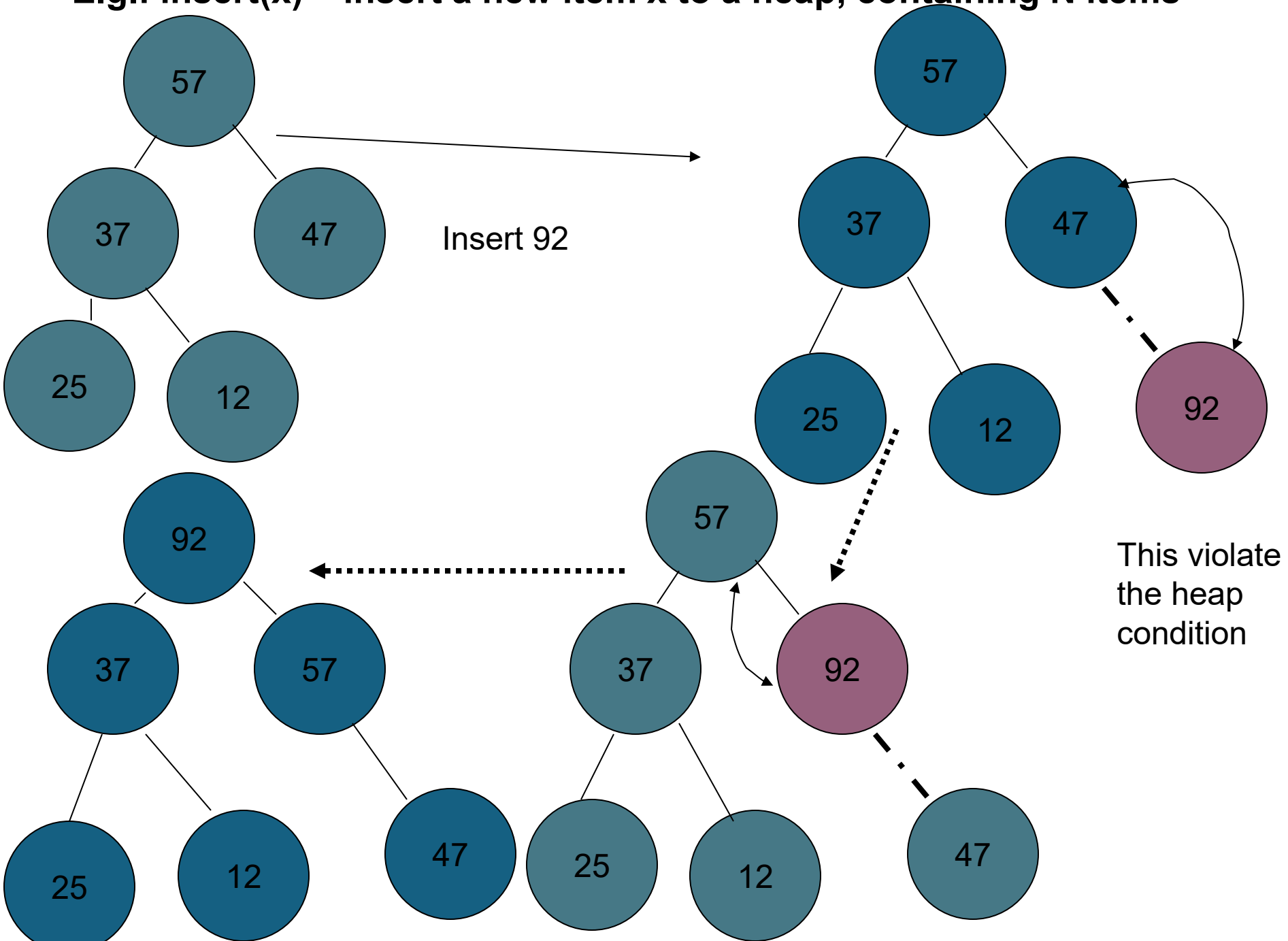


The tree may be easily constructed from the array by noticing that the left child of the k as 2k and right child 2k+1.

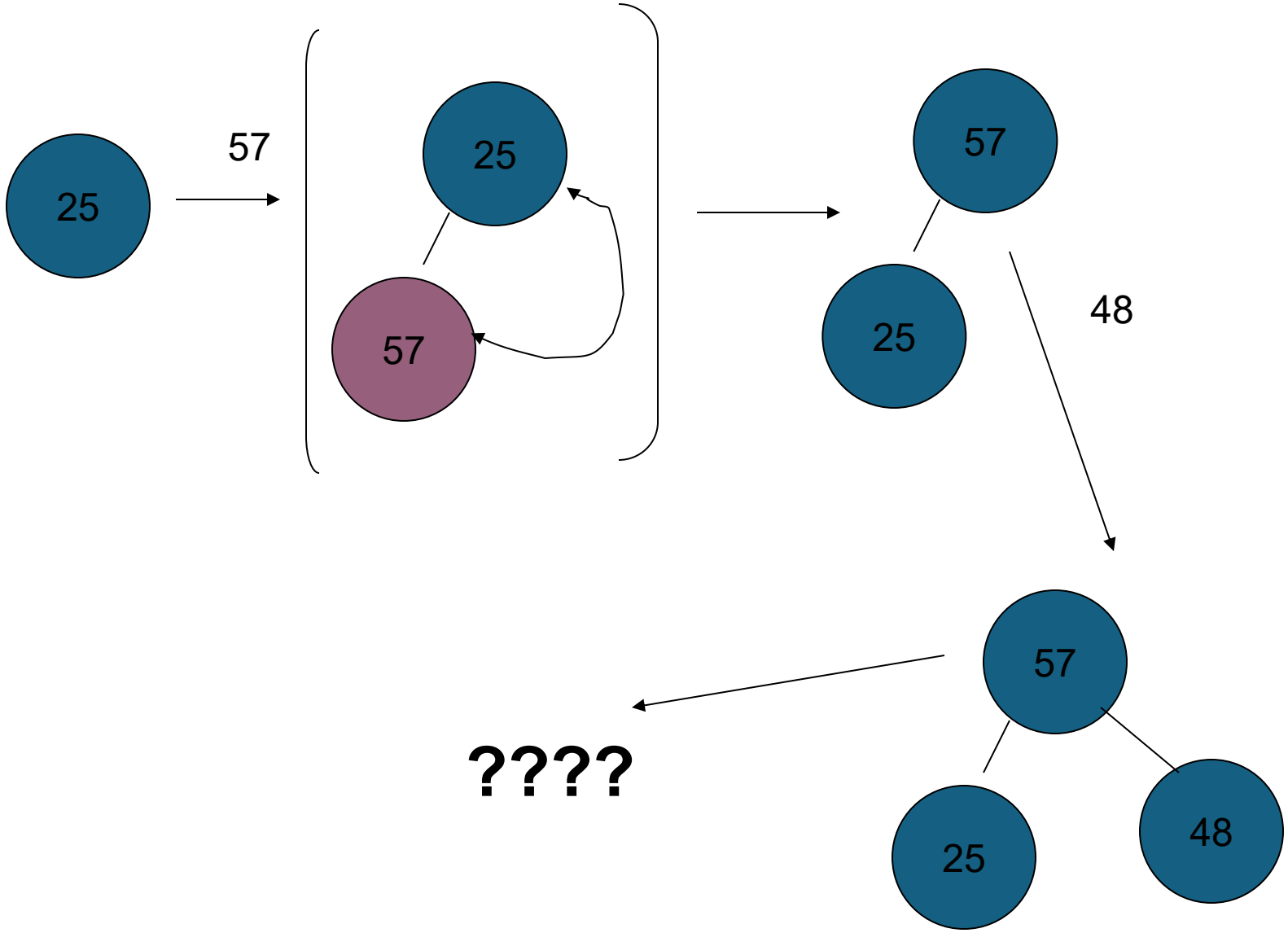
Algorithms on Heaps

- The algorithms on heap works by first making a simple structural modification which could violate the heap condition, the traveling through the heap modifying it to ensure that the heap condition is satisfied everywhere.
- Some of the algorithm travel through the heap from bottom to top, others top to bottom.

E.g.: insert(x) – insert a new item x to a heap, containing N items



Creating a Heap of size 8. The original file is 25,57,48,37,12,92,86,33

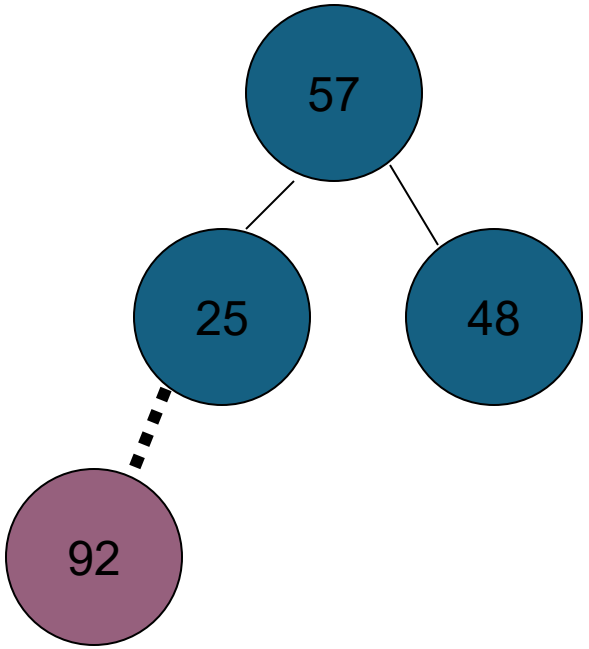


- We want to build and maintain a data structure containing records with numerical keys (priorities) and supporting some of the following operations.
- Construct a priority queue from given N items
- Insert a new item (with a priority)
- Replace the largest item with a new item.(unless new item is larger)
- Change the priority of an item.

- To be able build a heap, it is necessary, first implement the insert operation. This operation will increase the size of the heap by one. N must be incremented. Then the record to be inserted is put into A[N] , but this may violate the heap property (ie. New node is grater than its parent) then the violation can be fixed by exchanging new node with its parent .

- The code for this method is straight forward.
- Insert : add a new item to a[n], then upheap[n] fix the heap condition violation at N.

Pseudo code for insertion



Insert (X)

N++

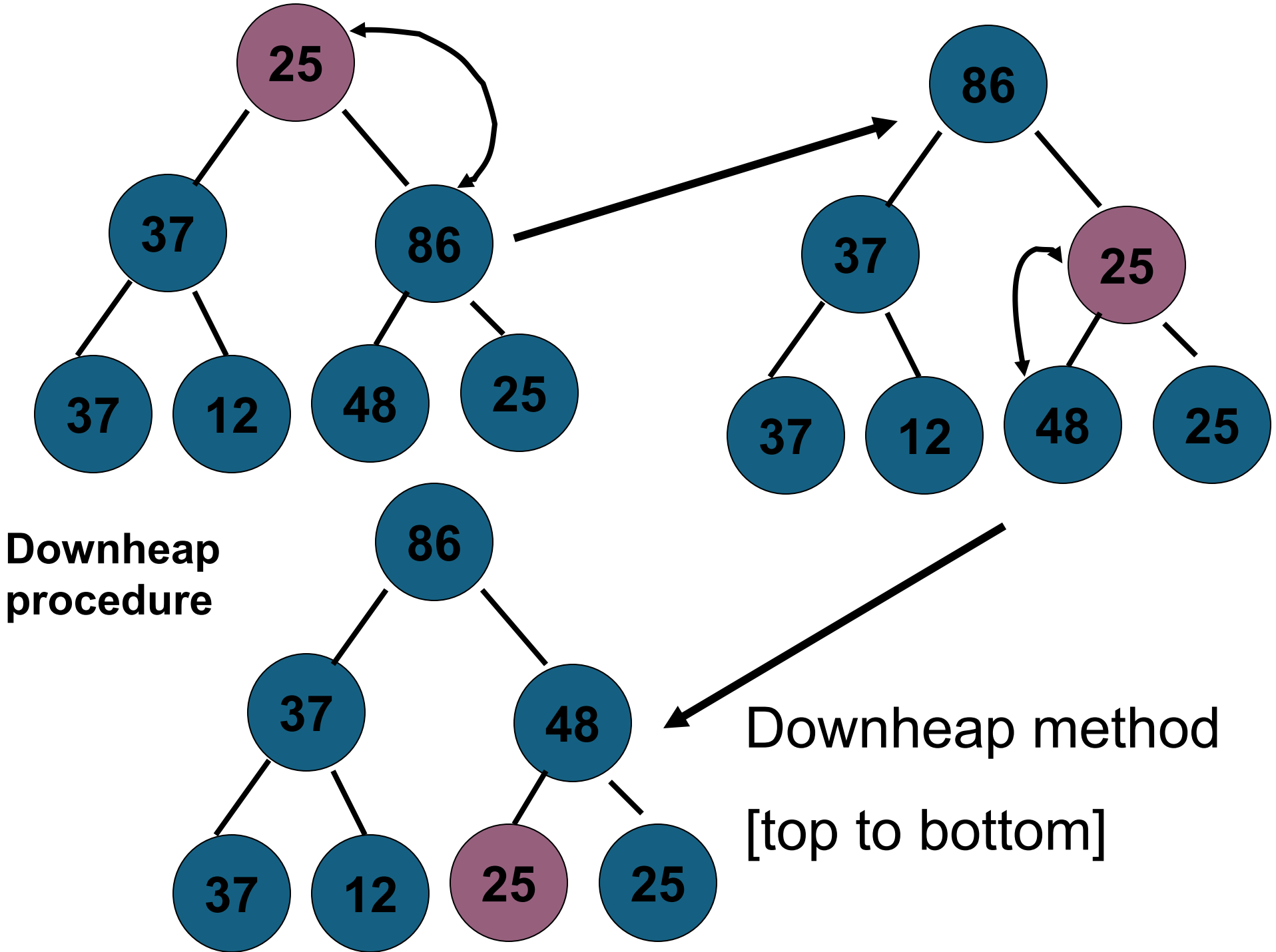
A[N]=x

Upheap{N]

Upheap procedure ??

Downheap Method

- If the element at node k is smaller than its children, that element is moved down the heap without violating the heap property at any nodes touched.


25
37
86
37
12
48
25
86
37
25
37
12
48
25
Downheap procedure
86
37
48
37
12
25
25
Downheap method
[top to bottom]

Remove the largest item-pseudo code

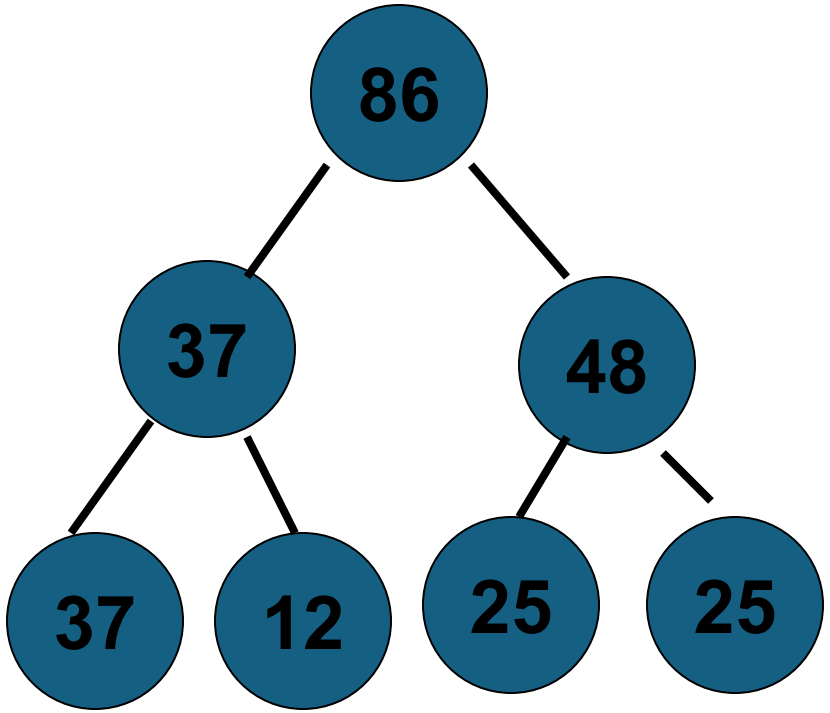
Remove the highest priority element ( ) ;

return = A[0];

A[0] = A[N-1];

N--;

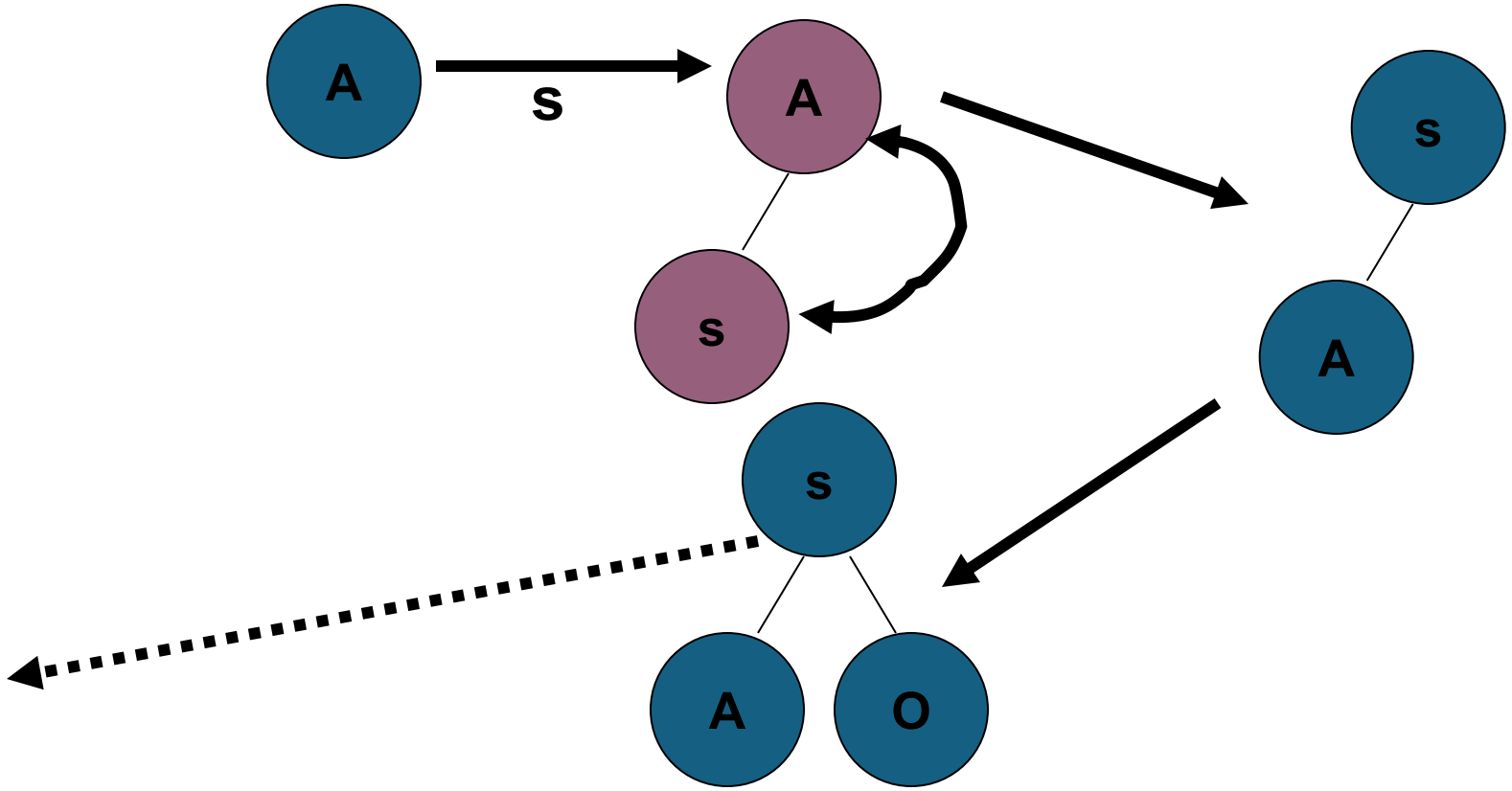
Upheap[N];



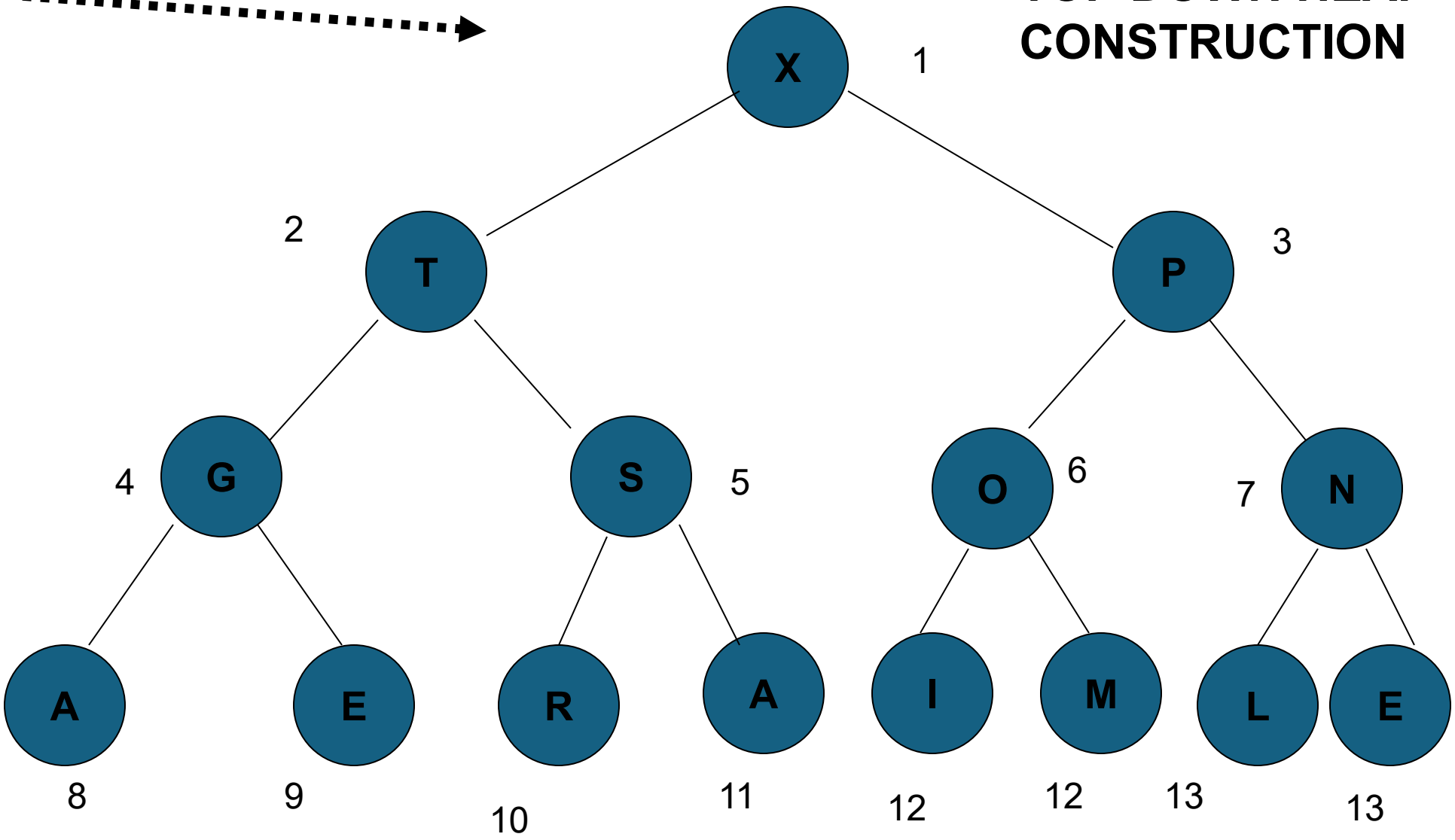
The return value is set from A[0] and then the element from A[N-1] is put into A[0] and the size of the heap is decremented, call down heap procedure to fix up the heap condition everywhere.

Heap Sort

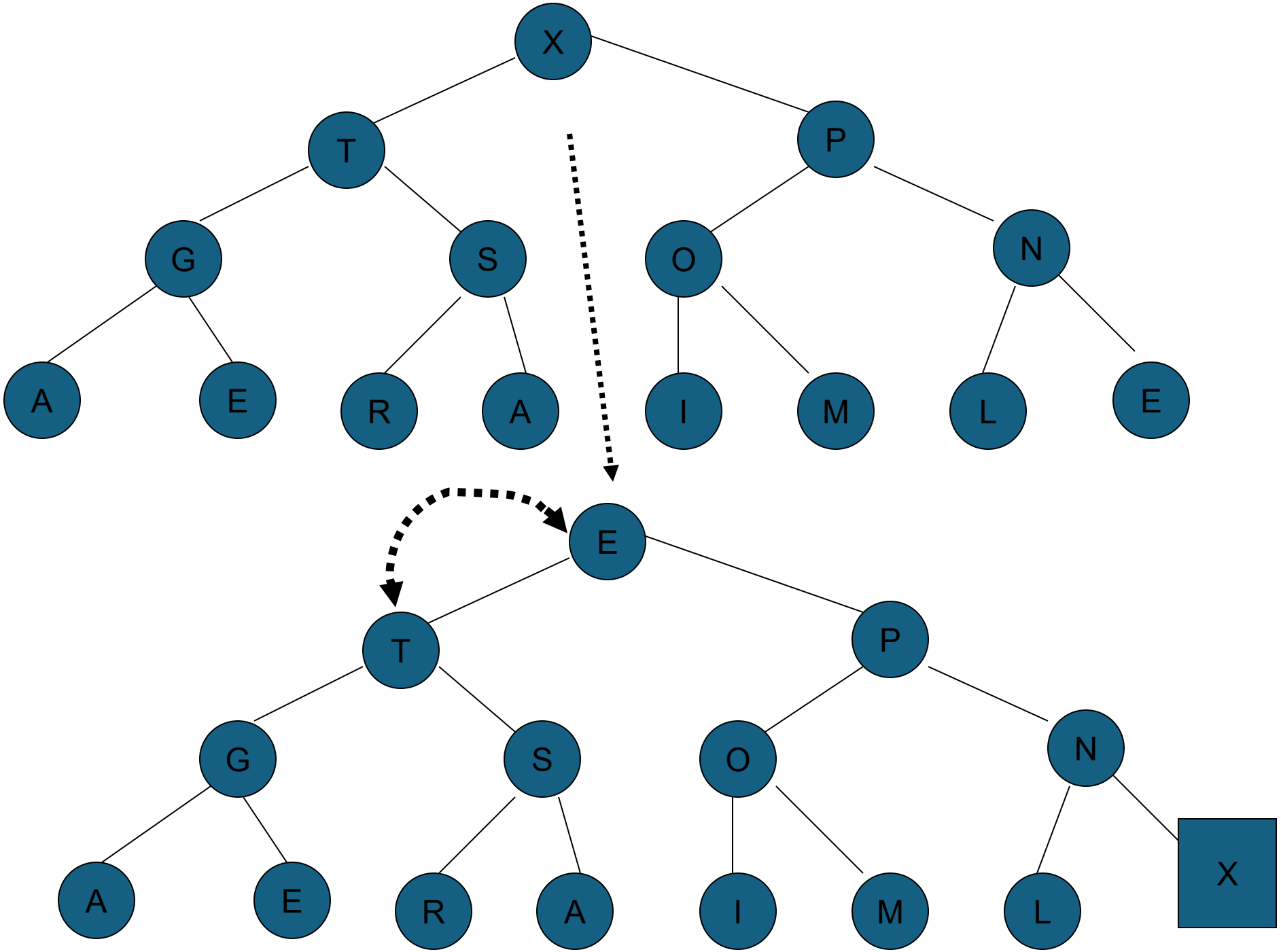
Eg : A SORTING EXAMPLE

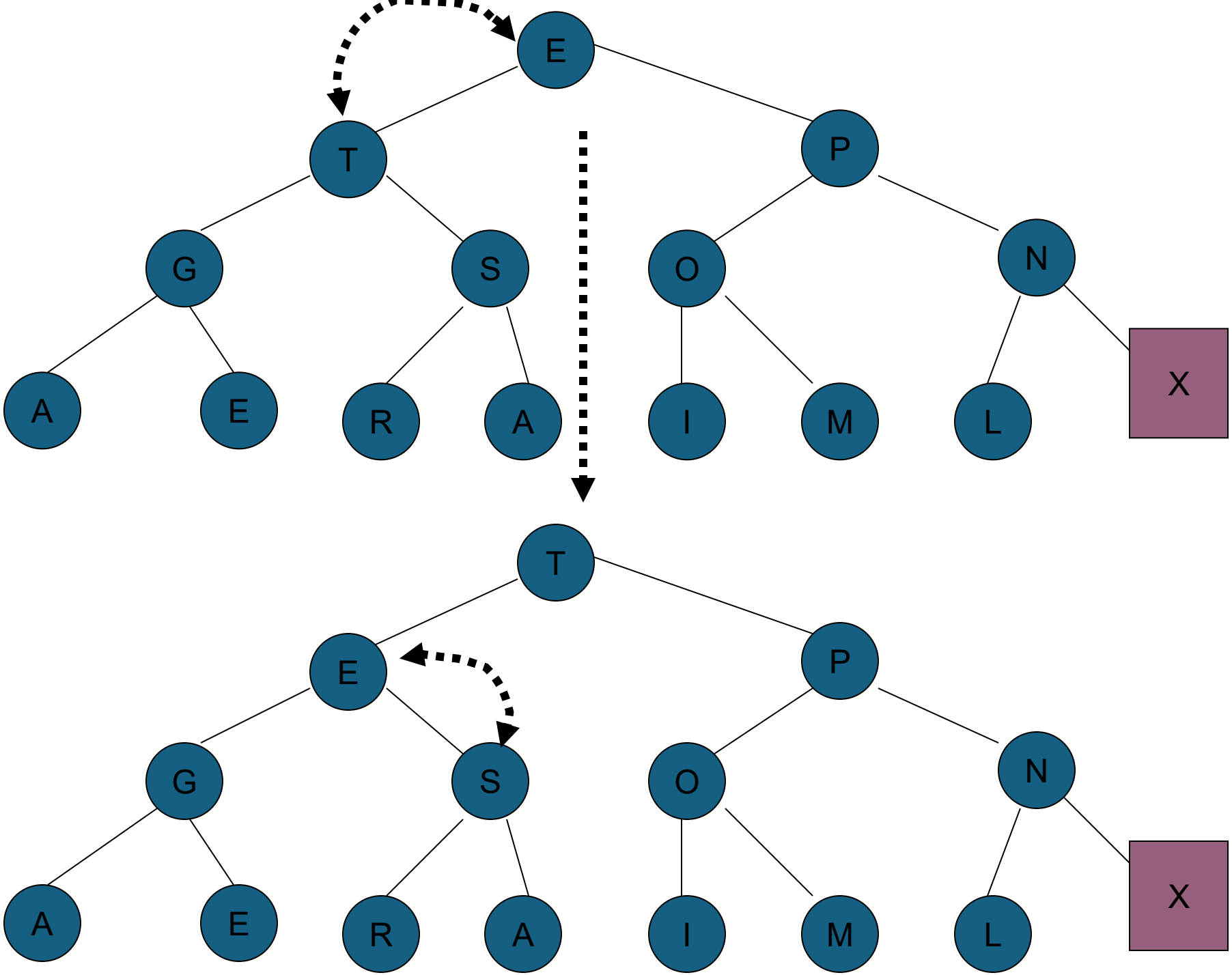


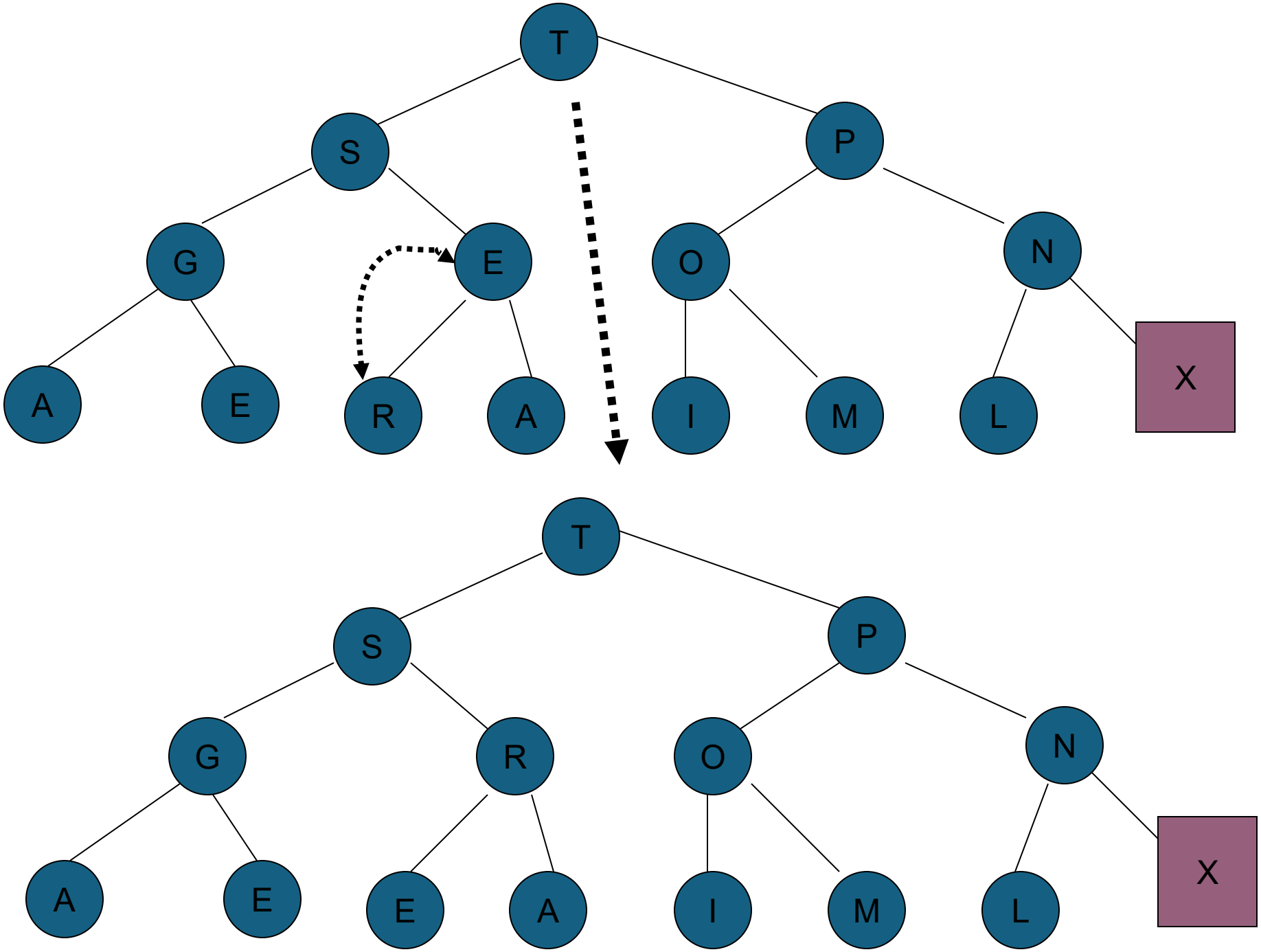
TOP DOWN HEAP CONSTRUCTION

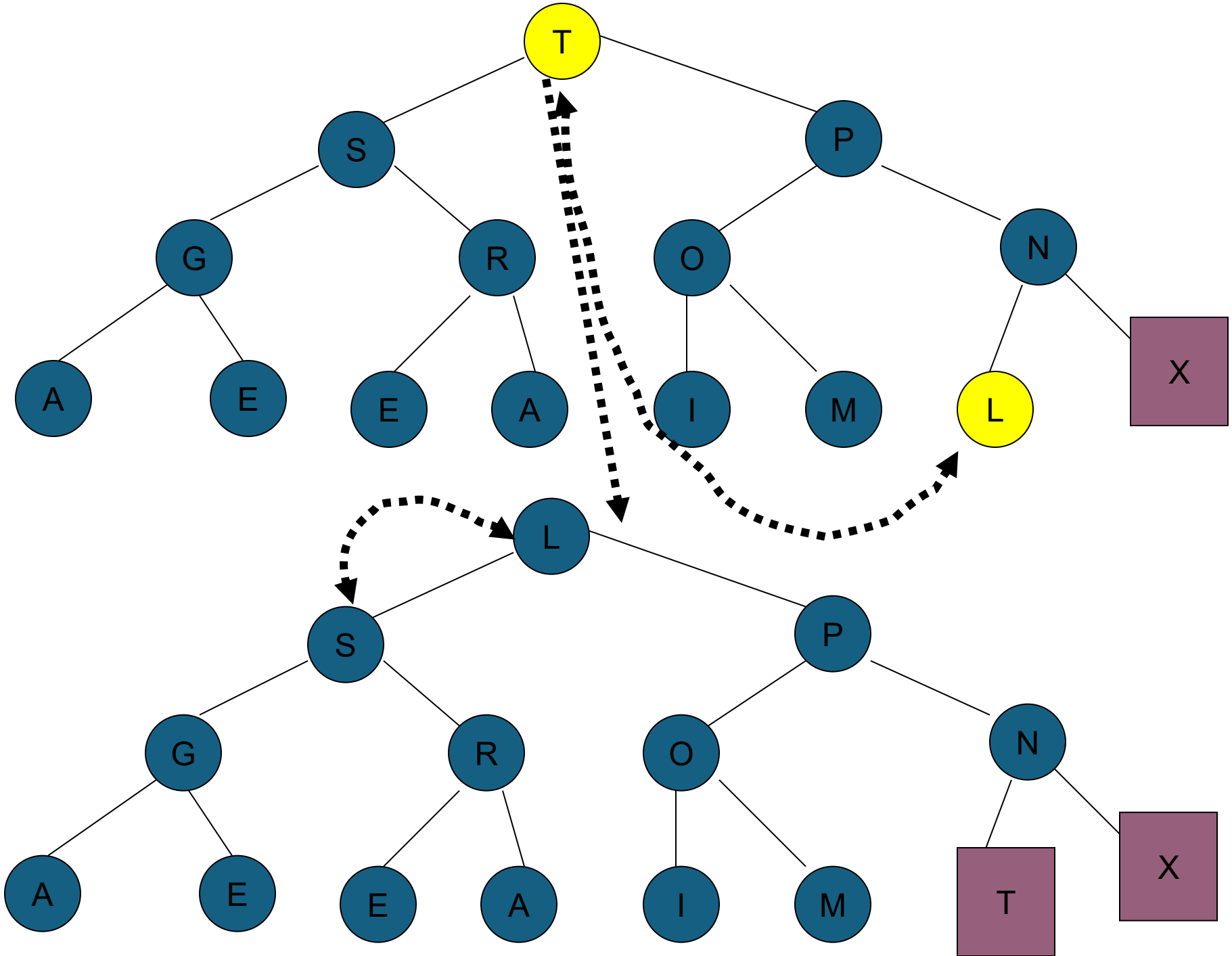


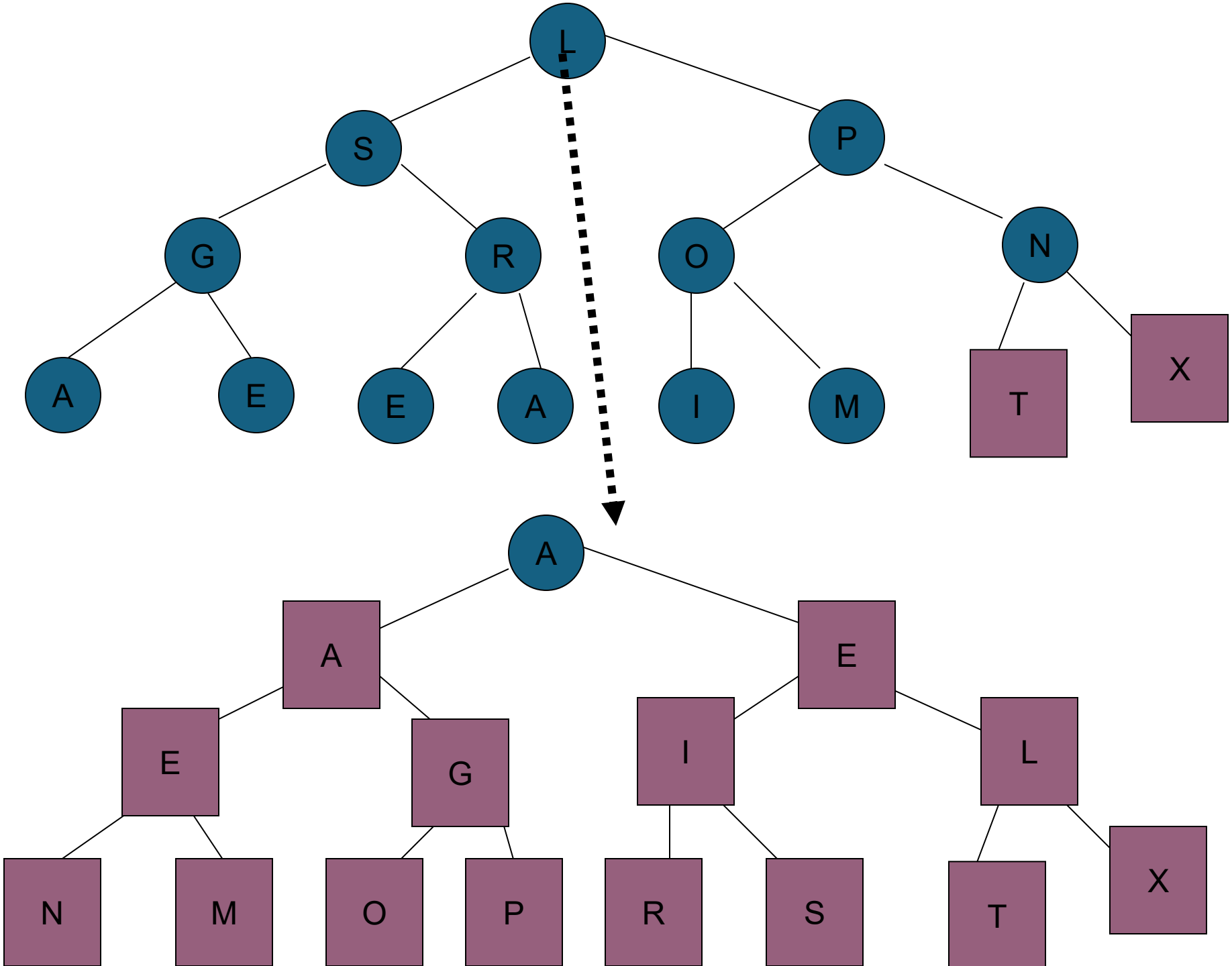
Above figure shows the heaps constructed as the keys A SORTING EXAMPLE are inserted in that order into an initially empty heap.


X
T
P
G
S
O
N
A
E
R
A
I
M
L
E
E
T
P
G
S
O
N
A
E
R
A
I
M
L
X


E
T
P
G
S
O
N
A
E
R
A
I
M
L
X
T
E
P
G
S
O
N
A
E
R
A
I
M
L
X


T
S
P
G
E
O
N
A
E
R
A
I
M
L
X
T
S
P
G
R
O
N
A
E
E
A
I
M
L
X


T
S
P
G
R
O
N
A
E
E
A
I
M
L
X
L
S
P
G
R
O
N
A
E
E
A
I
M
T
X


L
S
P
G
R
O
N
A
E
E
A
I
M
T
X
A
A
E
E
G
I
L
N
M
O
P
R
S
T
X

- 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
- A[I] A A E E G I L N M O P R S T X

- We were already noted that remove can be implemented by exchanging the first and last element and decrementing n by one and calling downheap from one.

Lecture – 8
Advanced Sorting Methods –Part E

M.G.Noel A.S Fernando (PhD)

Professor

Dept. of Information Systems Engineering, UCSC

Counting Sort Algorithm

- **Counting Sort** is a **non-comparison-based** sorting algorithm.
- It is particularly efficient when the range of input values is small compared to the number of elements to be sorted.
- The basic idea behind **Counting Sort** is to count the **frequency** of each distinct element in the input array and use that information to place the elements in their correct sorted positions.
- For example, for input [1, 4, 3, 2, 2, 1], the output should be [1, 1, 2, 2, 3, 4]
- The count is stored in an auxiliary array and the sorting is done by mapping the count as an index of the auxiliary array.

Working of Counting Sort

Step 1: Find out the maximum element (let it be max) from the given array.

	0	1	2	3	4	5	6	7	Max
arr[]=	2	5	3	0	2	3	0	3	5

Step 2: Initialize a cntArr[] of length max+1 with all elements as 0. This array will be used for sorting the occurrences of the elements of the input array.

	0	1	2	3	4	5
cntArr[]=	0	0	0	0	0	0

Working of Counting Sort cont...

- Step 3: In the cntArr[], store the count of each unique elements of the input array at the respective indices

	0	1	2	3	4	5	6	7
arr[]=	2	5	3	0	2	3	0	3

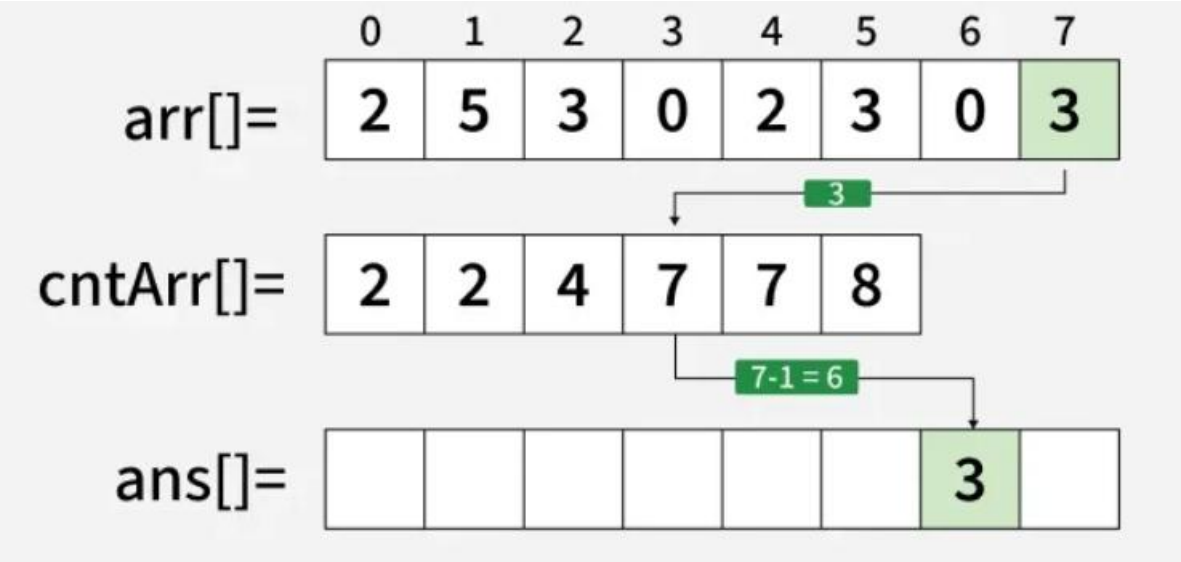
	0	1	2	3	4	5
cntArr[]=	2	0	2	3	0	1

- Step 4: Store the cumulative sum or prefix sum of the elements of the cntArr[] by doing cntArr[i-1]+cntArr[i]

	0	1	2	3	4	5
cntArr[]=	2	2	4	7	7	8

Working of Counting Sort cont....

- Step 5: Update ans [cntArr[arr[i]-1]=arr[i] and decrement cntArr[arr[i]].
- Traverse the input array in reverse to maintain the order of equal elements, ensuring the sort remains stable.

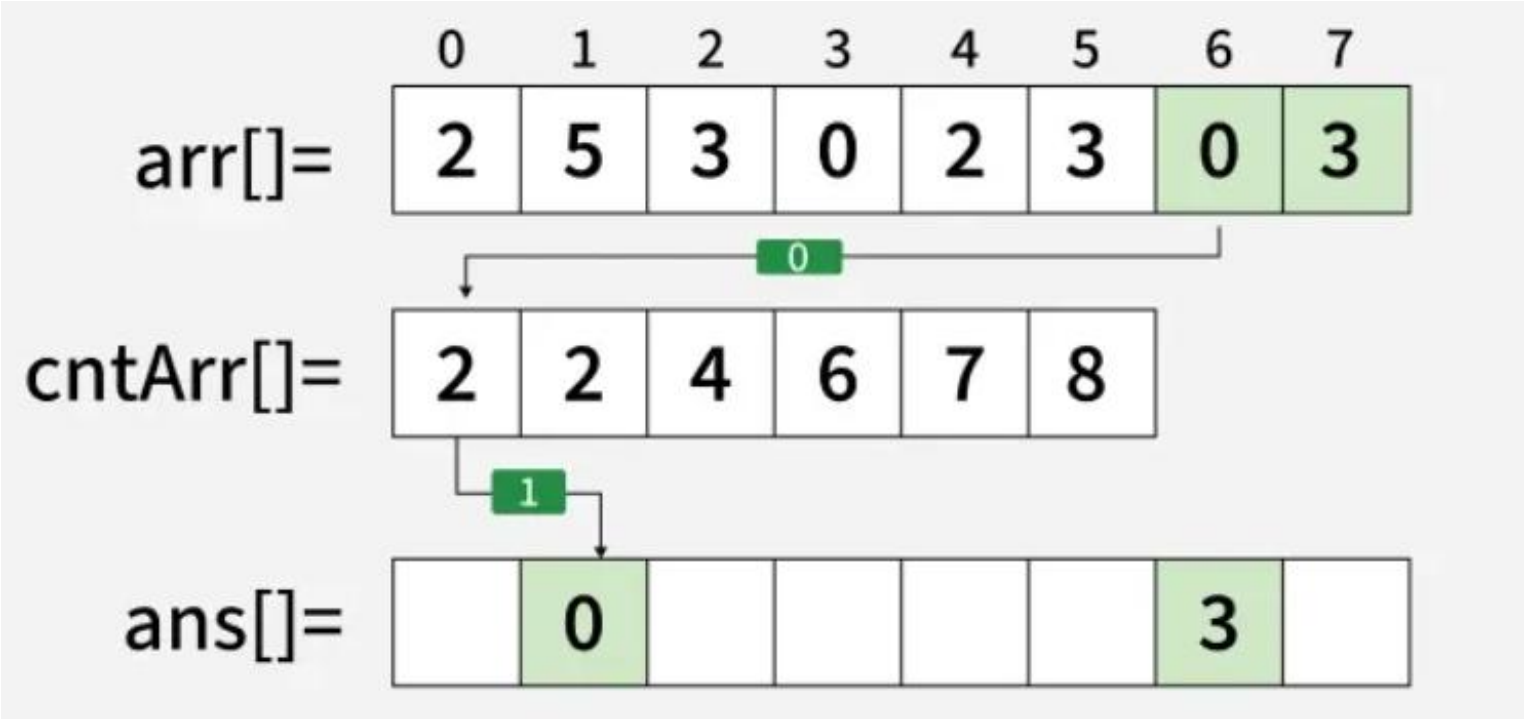


Original Array

Cnt Array

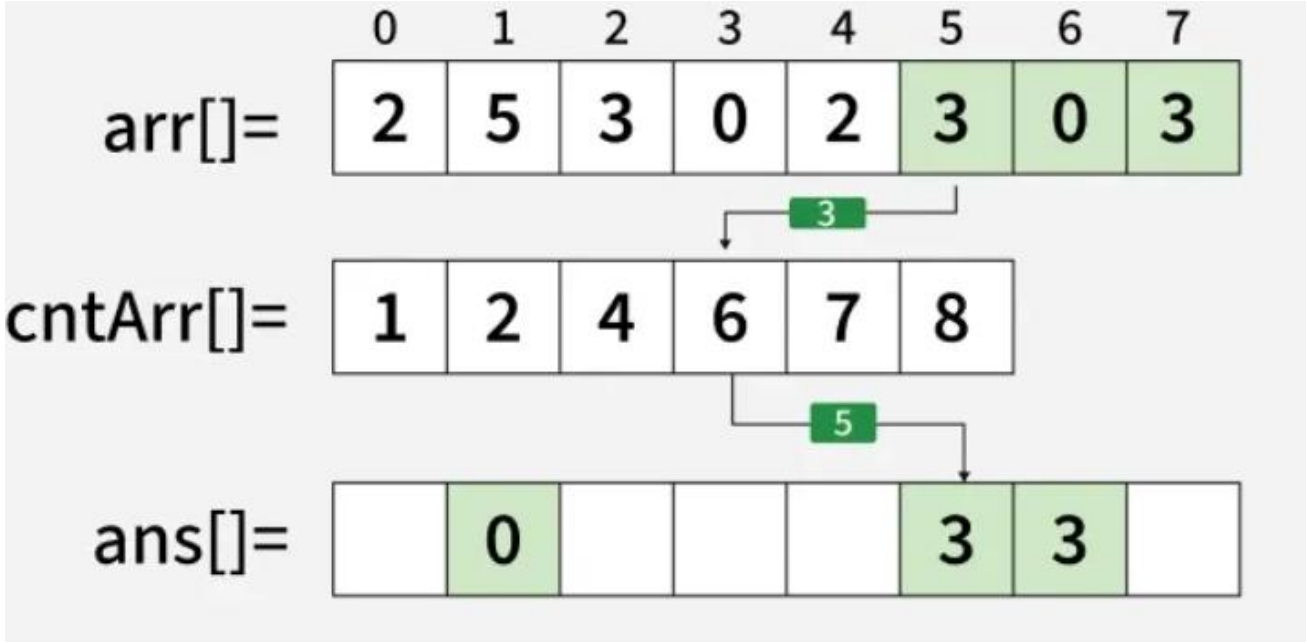
Working of Counting Sort cont....

- Step 6: update ans[cntArr[arr[6]]-1]=arr[6]
- Also, update cntArr[arr[6]]=cntArr[arr[6]....



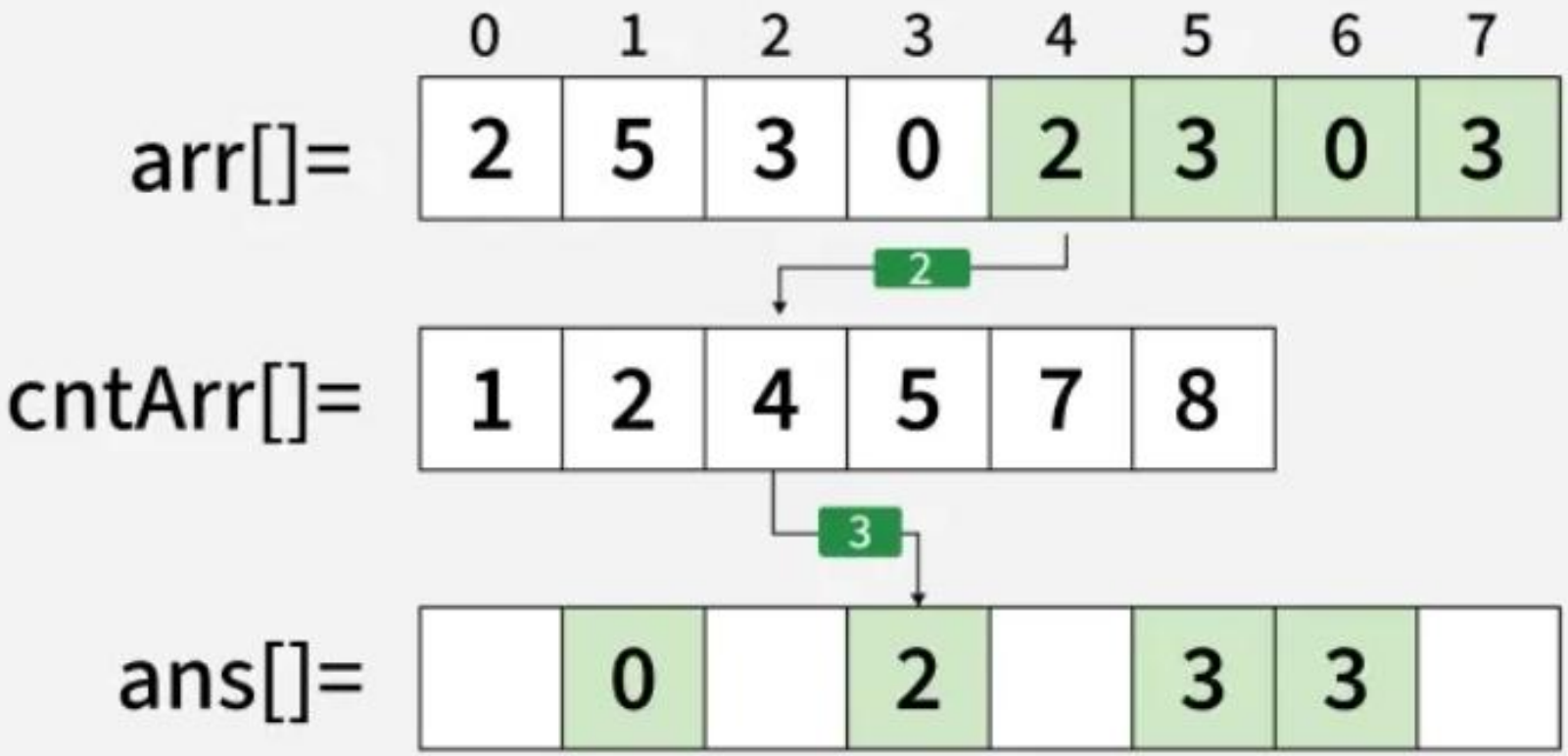
Working of Counting Sort cont....

- Step 7:nUpdate ans[cntArr[5]]-1]=arr[5]
- Also update cntArr[arr[5]]=cntArr[arr[5]...



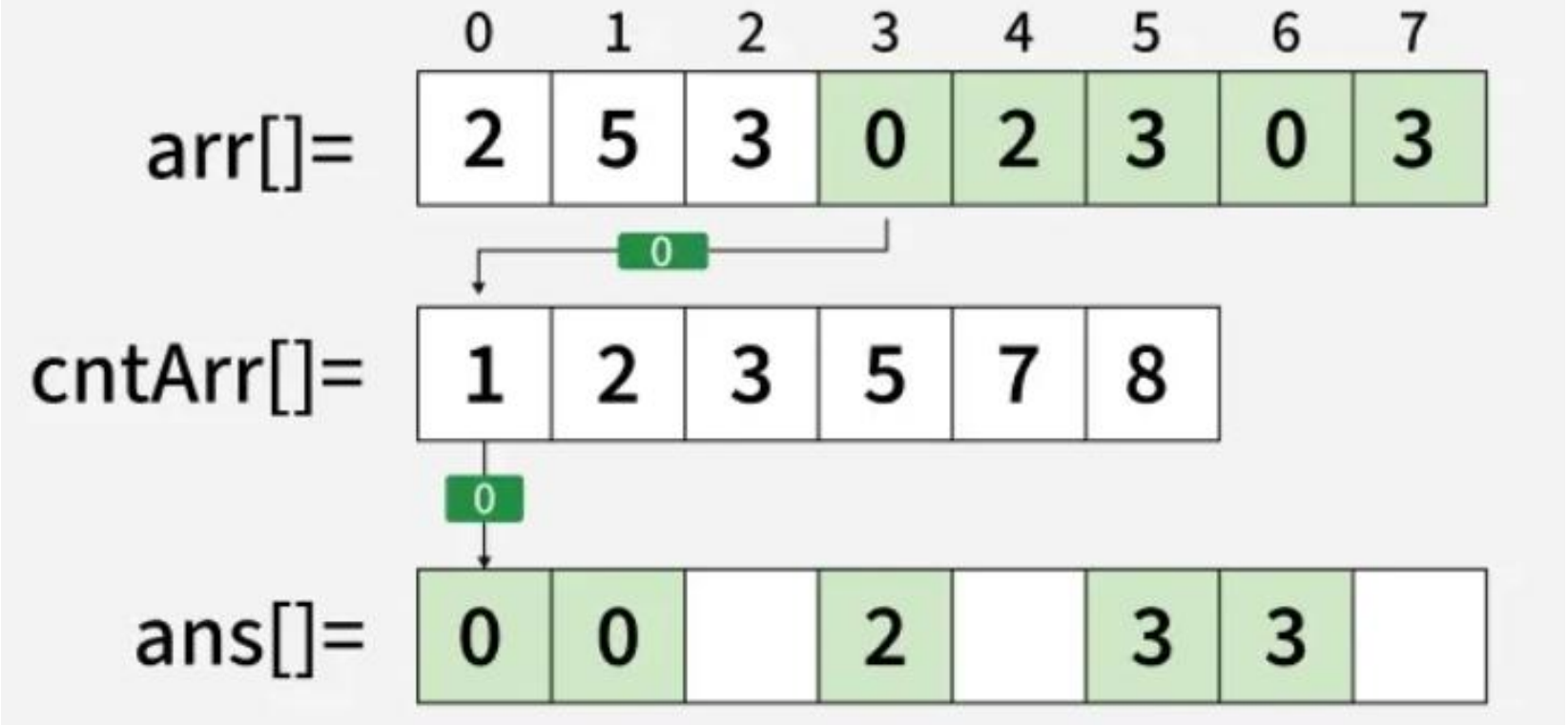
Working of Counting Sort cont....

Step 8 : Update ans[cntArr[arr[4]] - 1] =arr[4]
Also, update cntArr[arr[4]] = cntArr[arr[4]]- -



Working of Counting Sort cont....

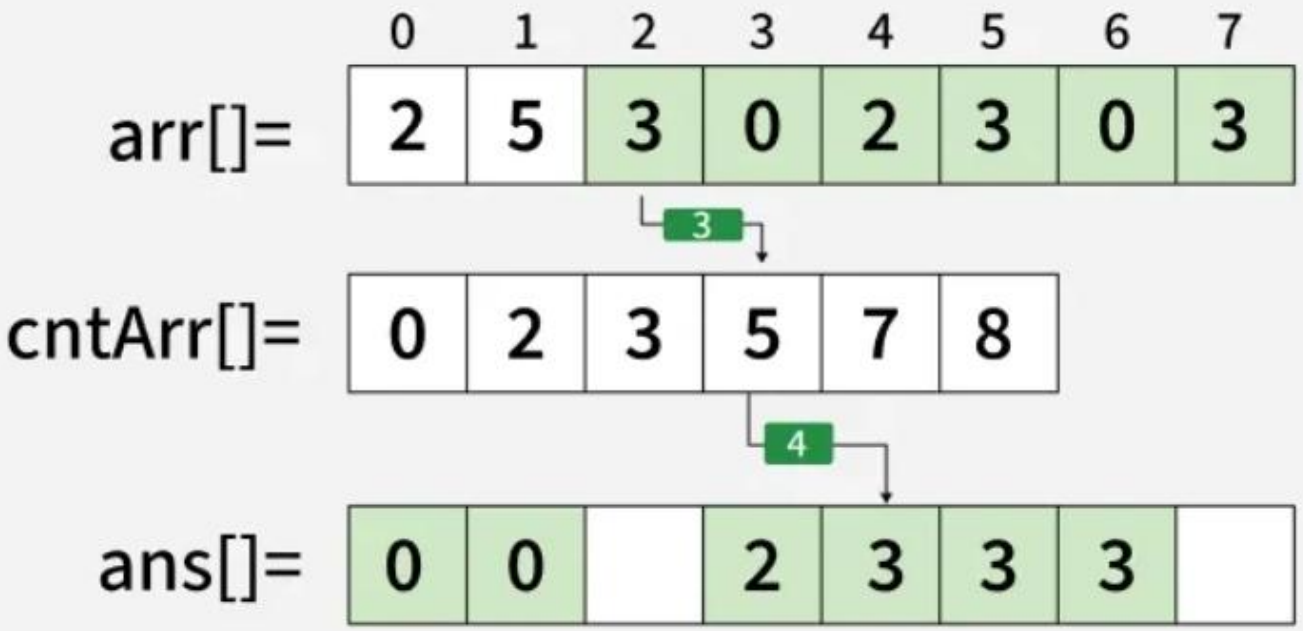
Step 9: Update ans[cntArr[arr[3]] - 1] =arr[3]
Also, update cntArr[arr[3]] = cntArr[arr[3]]- -



Working of Counting Sort cont....

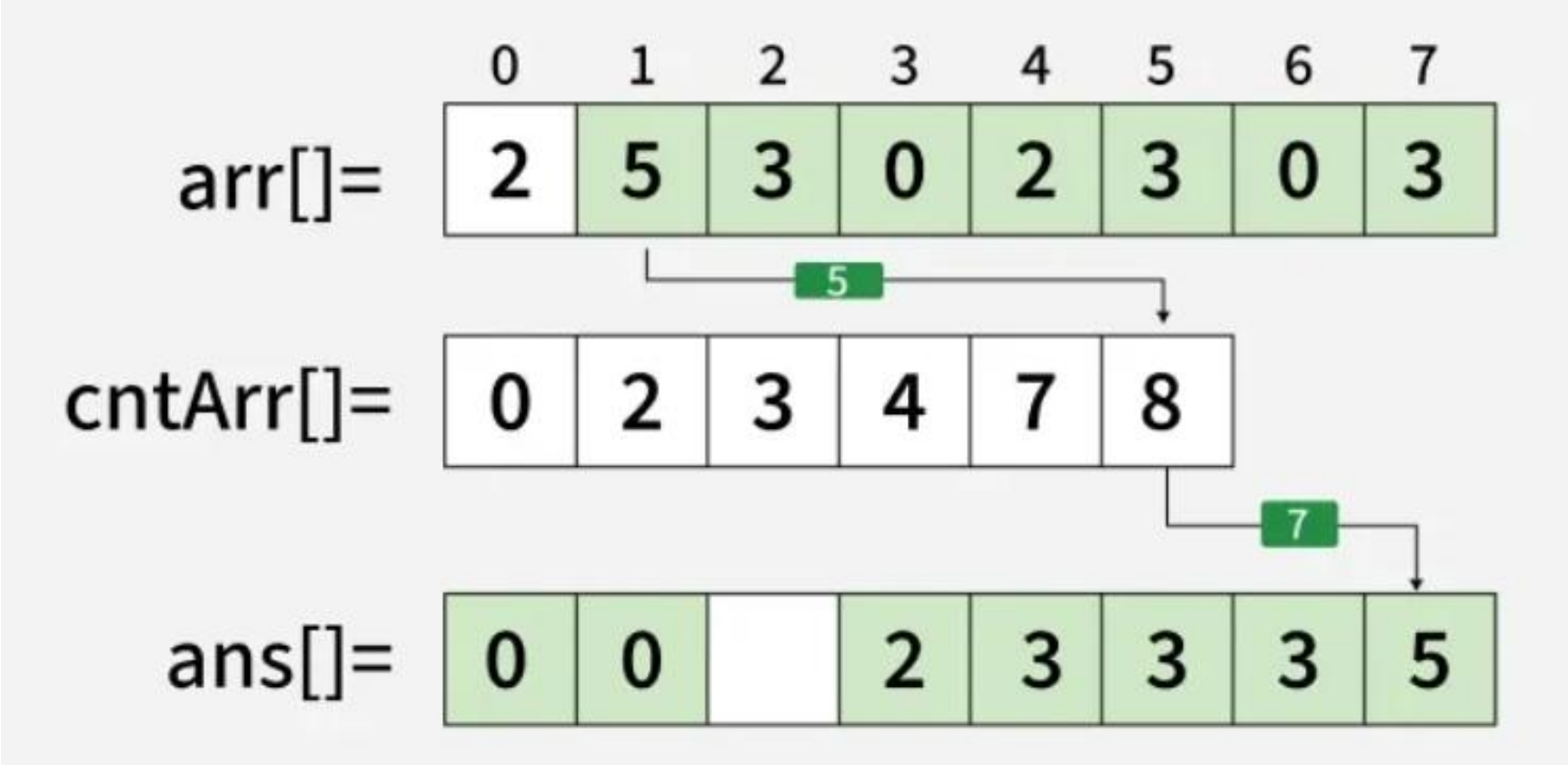
Step 10

Update ans[cntArr[arr[2]] - 1] =arr[2]
Also, update cntArr[arr[2]] = cntArr[arr[2]]- -



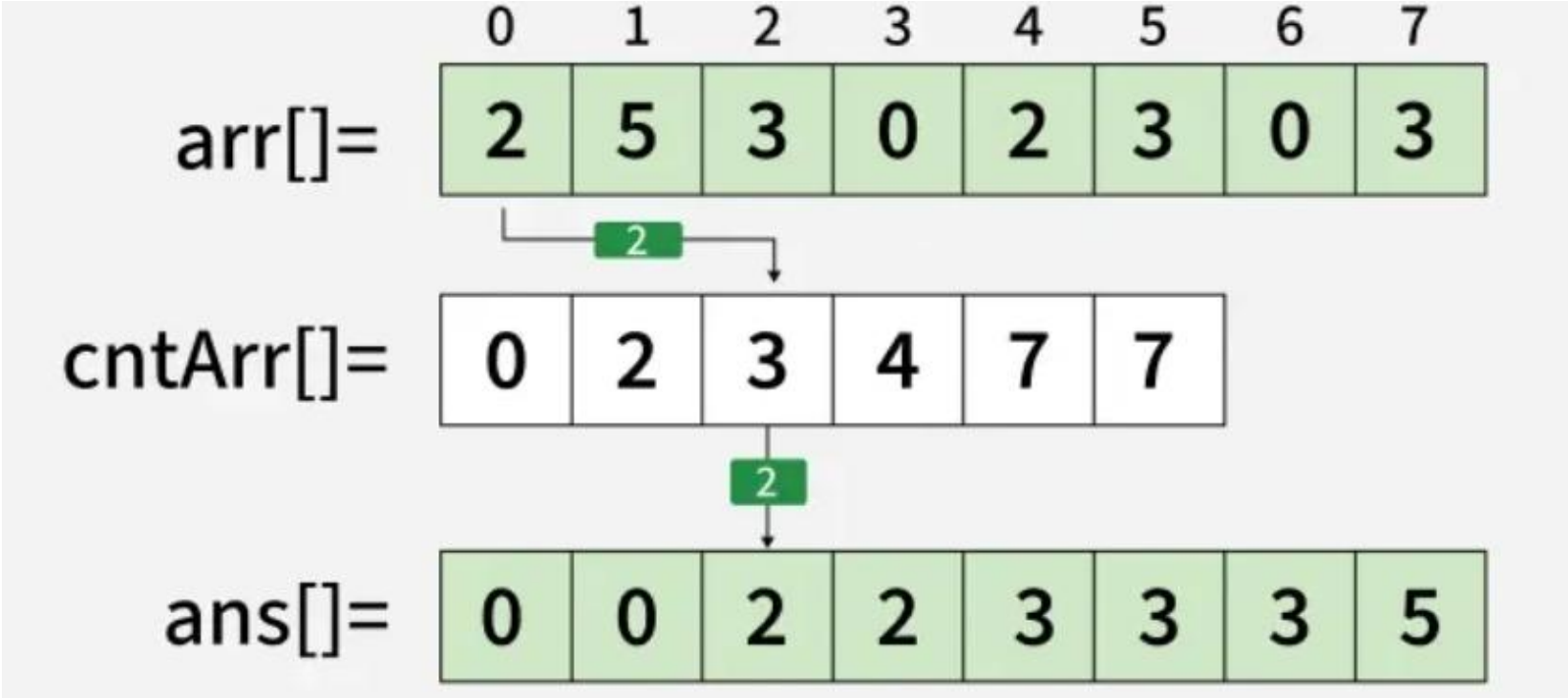
Working of Counting Sort cont....

- Step 11: Update ans[cntArr[arr[1]] - 1] =arr[1]
  Also, update cntArr[arr[1]] = cntArr[arr[1]]- -



Working of Counting Sort cont....

- Step 12: Update ans[cntArr[arr[0]] - 1] =arr[0]
  Also, update cntArr[arr[0]] = cntArr[arr[0]]- -



Counting Sort Algorithm

Step 1: Declare an auxiliary array **countArray[]** of size **max(inputArray[])+1** and initialize it with **0**.

Step 2: Traverse array **inputArray[]** and map each element of **inputArray[]** as an index of **countArray[]** array, i.e., execute **countArray[inputArray[i]]++** for **0 <= i < N**.

Step 3: Calculate the prefix sum at every index of array **inputArray**[].

Step 4 : Create an array **outputArray[]** of size **N**.

Step 5: Traverse array **inputArray[]** from end and update **outputArray[countArray[inputArray[i]] - 1] = inputArray[i]**. Also, update **countArray[inputArray[i]] = countArray[inputArray[i]]- -** .

C code - Counting Sort Algorithm

```
#include <stdio.h>
#include <stdlib.h>
void countSort(int inputArray[], int N) {
    // Finding the maximum element of
   // array inputArray[]
   int M = 0;
   for (int i = 0; i < N; i++)
      if (inputArray[i] > M)
         M = inputArray[i];
```

C code - Counting Sort Algorithm (Cont....)

```
// Initializing countArray[] with 0
  int* countArray = (int*)calloc(M + 1, sizeof(int));
   // Mapping each element of inputArray[]
  // as an index of countArray[] array
  for (int i = 0; i < N; i++)
     countArray[inputArray[i]]++;
     // Calculating prefix sum at every index
  // of array countArray[]
  for (int i = 1; i <= M; i++)
     countArray[i] += countArray[i - 1];
```

C code - Counting Sort Algorithm (Cont....)

```
// Creating outputArray[] from countArray[] array
  int* outputArray = (int*)malloc(N * sizeof(int));
  for (int i = N - 1; i >= 0; i--) {
     outputArray[countArray[inputArray[i]] - 1] = inputArray[i];
     countArray[inputArray[i]]--;
  }
    // Copying sorted elements back to inputArray[]
  for (int i = 0; i < N; i++)
     inputArray[i] = outputArray[i];
```

C code - Counting Sort Algorithm (Cont...)

```
// Freeing dynamically allocated memory
   free(countArray);
   free(outputArray);
}
// Driver code
int main() {
 // Input array
   int inputArray[] = {2,5,3,0,2,3,0,3};
   int N = sizeof(inputArray) / sizeof(inputArray[0]);
```

C code - Counting Sort Algorithm (Cont...)

```
// Sorting the array
   countSort(inputArray, N);
      // Printing the sorted array
   for (int i = 0; i < N; i++)
      printf("%d ", inputArray[i]);
      return 0;
}
```

Complexity Analysis of Counting Sort:

- **Time Complexity**: O(N+M), where **N** and **M** are the size of **inputArray[]** and **countArray[]** respectively.
  - Worst-case: O(N+M).
  - Average-case: O(N+M).
  - Best-case: O(N+M).
- **Auxiliary Space:** O(N+M), where **N** and **M** are the space taken by **outputArray[]** and **countArray[]** respectively.

Bucket Sort Algorithm

- Bucket Sort is a sorting algorithm that divides the unsorted array elements into several groups called buckets.
- Each bucket is then sorted by using any of the suitable sorting algorithms or recursively applying the same bucket algorithm.
- Finally, the sorted buckets are combined to form a final sorted array.

Bucket Sort Algorithm:

- Create **n** empty buckets (Or lists) and do the following for every array element arr[i].
  - Insert arr[i] into bucket[n*array[i]]
  - Sort individual buckets using insertion sort.
  - Concatenate all sorted buckets.

Question 4

- Sort the following numbers using the Buket sort algorithm
- Numbers are :**0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68**
- Find the Worst-Case Time Complexity of the bucket sort

Radix Sort Algorithm

- Radix sort is the linear sorting algorithm that is used for integers.
- In Radix sort, there is digit by digit sorting is performed that is started from the least significant digit to the most significant digit.

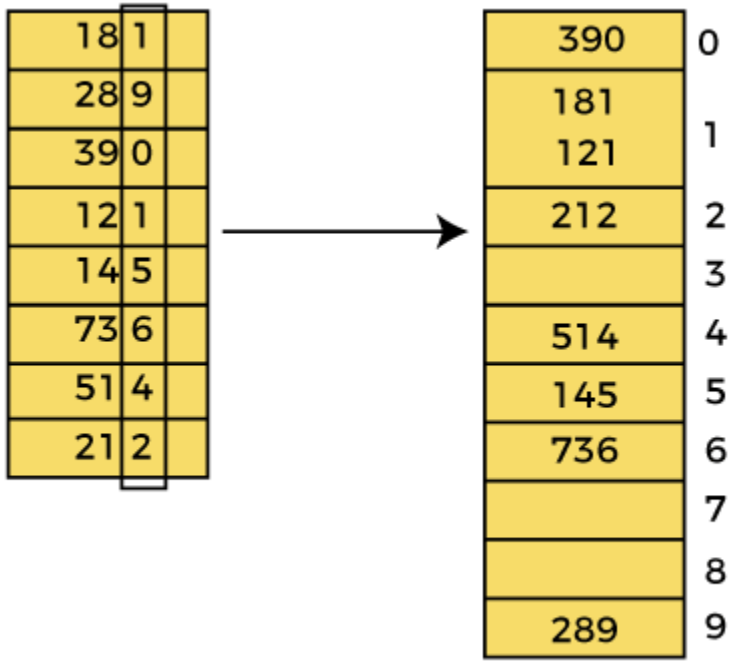
Working of Radix sort Algorithm

- Now let's see the working of radix sort in detail by using an example.
- To understand it more clearly, let's take an unsorted array and try to sort it using radix sort.
- It will make the explanation clearer and easier.

Working of Radix sort Algorithm

- Now, first sort the elements on the basis of unit place digits (i.e., **x = 0**).
- Here, we are using the counting sort algorithm to sort the elements.

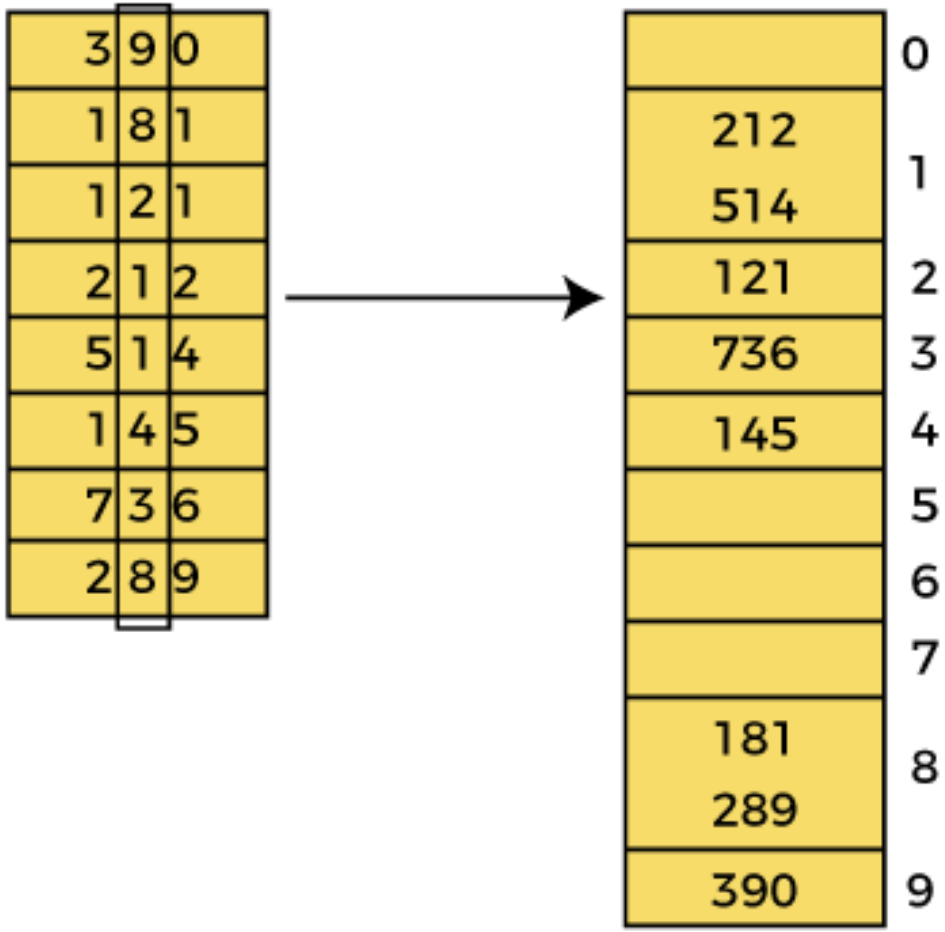
Pass 1: In the first pass, the list is sorted on the basis of the digits at 0's place.



After the first pass, the array elements are -

390	181	121	212	514	145	736	289

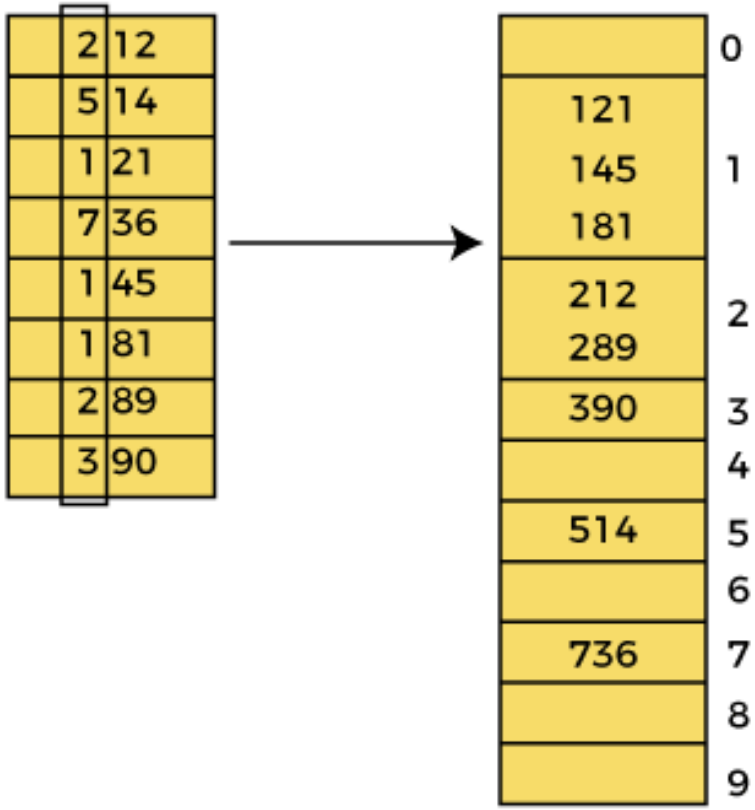
Pass 2: in this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 10th place)



After the second pass, the array elements are -

212	514	121	736	145	181	289	390

Pass 3: In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 100th place).



After the third pass, the array elements are -

121	145	181	212	289	390	514	736

Now, the array is sorted in ascending order.

Radix sort Algorithm

- Steps Involved in Radix sort to sort an array in ascending order are –
  - Find the maximum element of the array, let it be max
  - Find the number of digits in max, let it be k .
  - For each, i^{th} ranging from 1 To k , apply the counting sort algorithm for the ith least-significant digit of each element. If any element has less than i digits

  - Homework: coding , test run????

Question 6

- Explain the differences between the **Bucket Sort** algorithm and the **Radix Sort** algorithm.

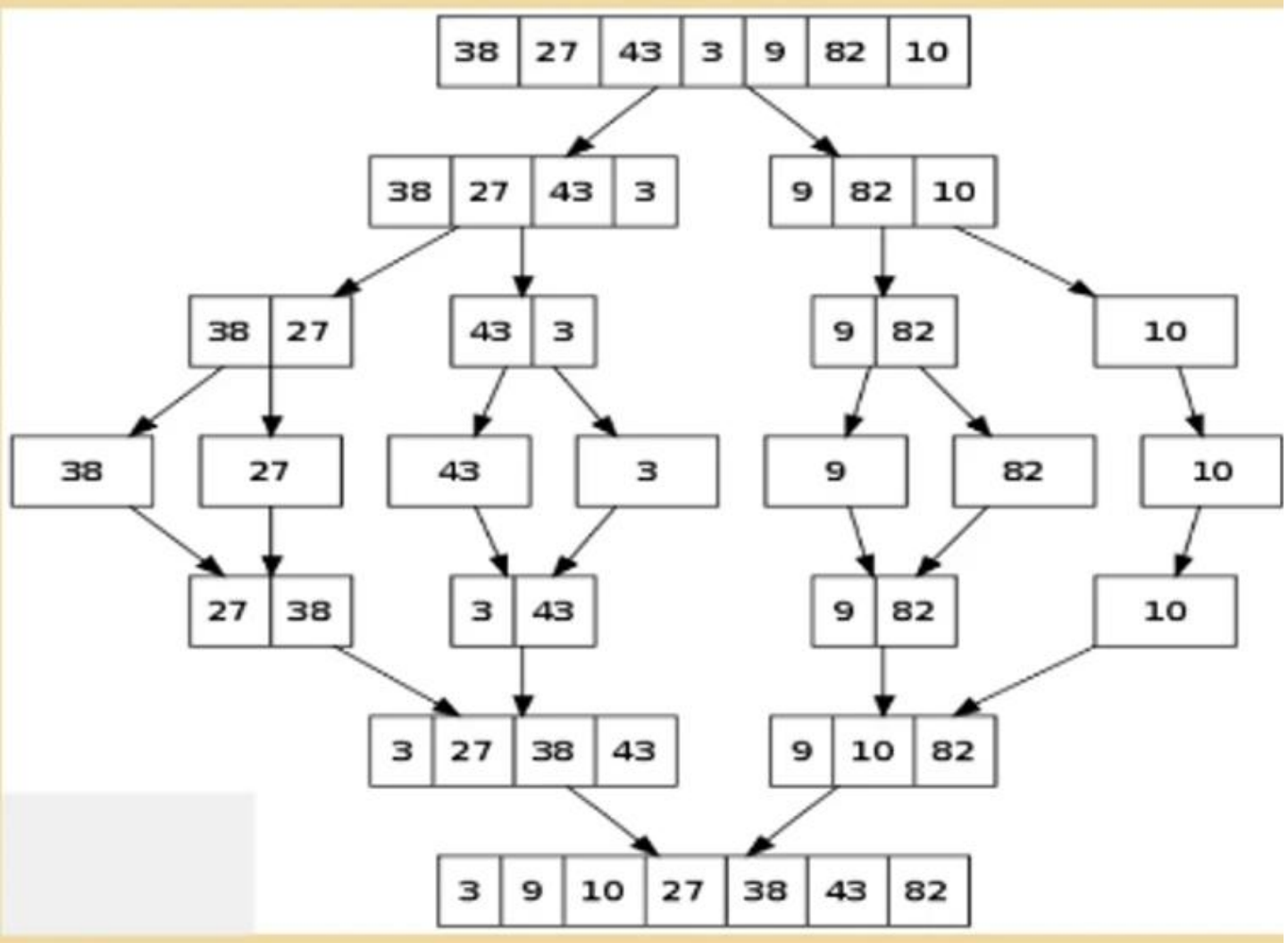
Q and A

Lecture 8

Queston 1

- Consider the following data set:
- 38,27,43,3,9,82,10
- Using a suitable diagram, explain how to sort the above dataset using the **Merge Sort algorithm**.
- Write C code to implement the Merge Sort algorithm

Example – Merge Sort



C Code – Merge sort

```
#include <stdio.h>
// Function to merge two halves
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;  // Size of left half
    int n2 = right - mid;     // Size of right half
    // Temporary arrays
    int L[n1], R[n2];
    // Copy data to temp arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
```

C Code – Merge sort Cont..

```
// Merge the two halves back into arr[]
  int i = 0, j = 0, k = left;
  while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
      arr[k] = L[i];
      i++;
    } else {
      arr[k] = R[j];
      j++;
    }
    k++;
  }
}
```

C Code – Merge sort Cont..

```
// Recursive merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}
```

C Code – Merge sort Cont..

```
// Utility function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

C Code – Merge sort Cont..

```
// Main function
int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int size = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: \n");
    printArray(arr, size);
    mergeSort(arr, 0, size - 1);
    printf("Sorted array: \n");
    printArray(arr, size);
    return 0;
}
```

Question 2

a)Sort the following dataset using the **Quick Sort algorithm**:
25, 57, 48, 37, 12, 92, 86, 33

- **Note:** Use the **first element** as the pivot at all stages.

b) write pseudocode algorithm for the quick sort

Sorting Process

- Eg : 25,57,48,37,12,92,86,33
- If we choose x[0] as pivot value and store it in position j
- Then 1st element 25 is placed in its proper position
- 12,**25**,57,48,37,92,86,33
- Nothing need to be done to sort the first of these arrays: file of one element is already sorted.
- To sort the 2nd sub array/list , the process is repeated and the sub array is further sub-divided.

Sorting Process

- (12),**25**,(57,48,37,92,86,33)
- If we choose 57 as a pivot then
- (12),**25**,(48,37,33),57,(92,86)
- If we choose 48 and 92 as pivot.
- (12),**25**, (37,33),48,57,92,(86)
- If we select 37 as pivot
- 12,25,33,48,57,86,92
- The final array/list is sorted.

Quicksort-pseudo code algorithm

- Quicksort Algorithm

```
//Sort the array elements A[first] throught A[last]
Input: Array A, int first, int last
Output:
quicksort(A, first, last){
   if(first < last){
        Choose a pivot
        Partition the array about the pivot
        pivotIndex = index of the pivot
        quicksort(A, first, pivotIndex-1)
        quicksort(A, pivotIndex+1, last)
   }
}
```

Partition Algorithm

- Two references **up** and **down** are moved towards each other in the following fashion.
- **Algorithm**
- **Step 1**: Repeatedly increase the pointer **down** by one position until **x[down] >a**
- **Step 2**: Repeatedly decrease the pointer **up** by one position until x[up] =< a
- **Step 3**: if up > down then interchange **x[down] with x[up]**
- The process is repeated until the condition in step 3 fails (ie up =< down) at that point **x[up]** is interchanged with **x[lb]** (which equal to a), whose final position was sought and **j** is set to **up**.

Question 3

- Apply the above portion algorithm portion the following data set:
- 25 57 48 37 12 92 86 33

Partition Process

down → 25 57 48 37 12 92 86 33 ← up 1st step

25 down → 57 48 37 12 92 86 33 ← up 1st step

25 down → 57 48 37 12 92 86 33 <--up 1st step fail

25 down → 57 48 37 12 92 86 <--up 33 2nd step

25 down → 57 48 37 12 92 <--up 86 33 2nd step

25 down → 57 48 37 12 <--up 92 86 33 2nd step fail

25 down → 12 48 37 57 <--up 92 86 33 ******transition

Partition Process

25 12 down → 48 37 57 <--up 92 86 33 ******

25 12 down → 48 37 57 <--up 92 86 33 1st step

25 12 48 down → 37 57 <--up 92 86 33 1st step fail

25 12 48 down → 37 57 <--up 92 86 33 2nd step

25 12 48 down → 37 57 <--up 92 86 33 2nd step

25 12 48 down → 37 <--up 57 92 86 33 2nd step

25 12 48 up down 37 57 92 86 33 2nd step

25 12 ← up 48 ← down 37 57 92 86 33 2nd step
fail

(12) 25 (48 37 57 92 86 33) **********transition

Question 4

- Sort the following numbers using the Buket sort algorithm
- Numbers are :**0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68**
- Find the Worst-Case Time Complexity of the bucket sort

Question 4

- Sort the following numbers using the Buket sort algorithm
- Numbers are :**0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68**
- **Step 1:** Create an array of size 10, where each slot represents a bucket.

0.78	0.17	0.39	0.26	0.72	0.94	0.21	0.12	0.23	0.68

Input Array

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

Bucket For Sorting Elements

Question 4

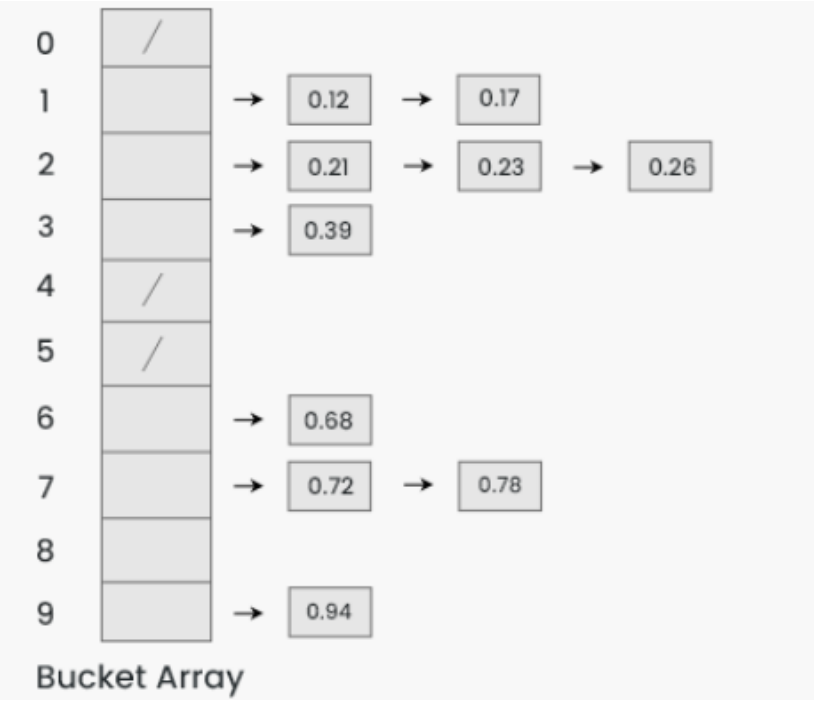
- **Step 2:** Insert elements into the buckets from the input array based on their range. (divide into 10 buckets)

For multiple elements on same bucket, Linked List Data Structure will be used

First bucket 0.01 to 0.09
Second bucket 0.10 to 0.19
And soon

Question 4

- **Step 3:** Sort the elements within each bucket.
- Apply a stable sorting algorithm (e.g., Insertion Sort) to sort the elements within each bucket.

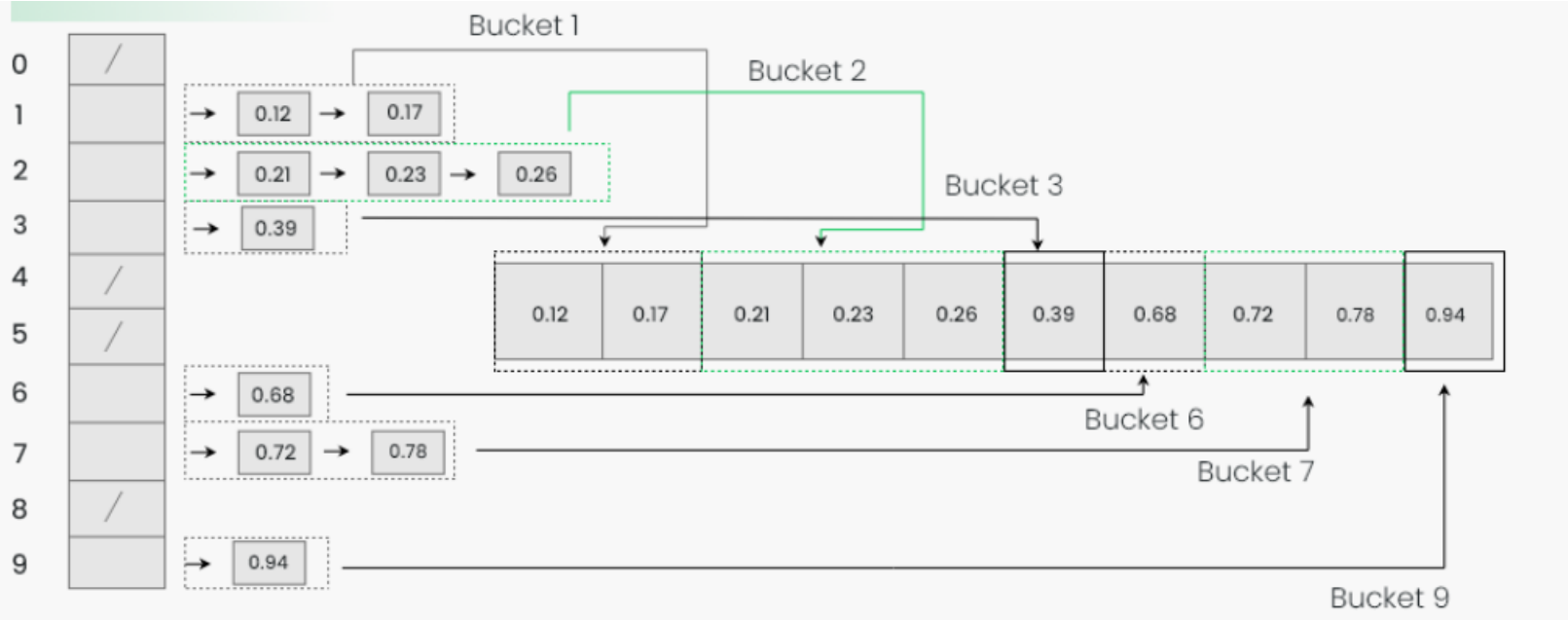


Each Bucket is treated a different Linked List and Sorted in ascending order

*Any stable sorting algorithm can be used for this purpose

Question 4

Step 4: Inserting buckets in ascending order into the resultent array



Question 4

Step 5 : Return the Sorted Array

0.12	0.17	0.21	0.23	0.26	0.39	0.68	0.72	0.78	0.94

Complexity Analysis of Bucket Sort Algorithm

- **Worst Case Time Complexity:** $O(n^2)$ The worst case happens when one bucket gets all the elements. In this case, we will be running insertion sort on all items which will make the time complexity as $O(n^2)$

Question 5

(a) (i) A priority queue is implemented as a Max-Heap. Initially, it has 5 elements. The level-order traversal (traversal and filling from left to right level by level) of the heap is: 10, 8, 5, 3, 2.

(ii) Two new elements 1 and 7 are inserted into the heap in the given order. What would be the level-order traversal of the heap after the insertion of the two new elements? Your answer should include insertion and fixing processes using step by step approach.

(b)(i) If one wants to delete the root value from the max-heap created in part (a)(ii) above. How could you eliminate the heap property violation which arises, using the top-down approach through fixing process without using an additional array?

(ii)How could you extend the methodology described in part b (i) above to sort all data values in the increasing order?

Question 6

- Explain the differences between the **Bucket Sort** algorithm and the **Radix Sort** algorithm.

Answer -Question 6

Aspect	Bucket Sort	Radix Sort
Basic Idea	Distributes the elements into a number of buckets, sorts each bucket (using another algorithm or recursively), and then concatenates the results.	Sorts numbers **digit by digit** (or bit by bit), starting from either the least significant digit (LSD) or most significant digit (MSD).
Type of Sorting	**Distribution sort** (based on value ranges).	**Digit-based sort** (based on positional representation of numbers).
Input Data	Works best when the input is **uniformly distributed** over a range (e.g., decimal fractions in [0, 1)).	Works best for **integers** or **strings** that can be broken into digits/characters.
Internal Sorting	Each bucket can be sorted using another algorithm (e.g., Insertion Sort or Quick Sort).	Uses a **stable sort** (like Counting Sort) to sort digits at each position.
Time Complexity	Average: (O(n + k)) (if elements are evenly distributed across buckets)Worst: (O(n^2)) (if all elements fall into one bucket).	(O(d \times (n + k))), where:(d) = number of digits(k) = range of each digit (e.g., 0–9 for decimal).
Stability	Depends on the sorting algorithm used inside buckets.	**Stable** if a stable algorithm is used for digit sorting (usually Counting Sort).
Example Use	Sorting floating-point numbers in [0, 1).	Sorting integers, phone numbers, or strings (e.g., “abc”, “bcd”).
Approach Summary	Divides data into *value-based buckets* → sorts each bucket.	Processes data *digit by digit* → sorts by each digit position.